Repair User Guide — Factory

Type 2: Not Under Warranty (Without Serial No.) — Full Flow

Source: Odoo Tour Recorder export — `Tour/1A. Repair User Guide - Factory [ Not Under Warranty (Without Serial No ) - Full Flow ].json`

Scenario: The product's warranty period has expired, and no serial number can be found on the item. This guide walks through creating the Helpdesk repair ticket through to final dispatch back to the customer.

Every screenshot below is from a single, clean, fully-passing automated run — the complete repair lifecycle, ticket creation through final dispatch, with zero manual steps or fallback/manual-verification screenshots. Real reference data throughout (customer `KURUNEGALA CASH CUSTOMER`, product `[JMC6] Multi Chopper 6Hp (Tractor Type)`, repair reason `Difficult to start the engine`, exact diagnosis values from `Data.txt`, spare part `[02BB 023] BALL BEARING 6202-2RS`, and Commercial Bank as the payment journal). Three real, previously-undiagnosed instance bugs were found and fixed along the way — see the QA notes below for the specifics.

1. Overview

This manual explains how to register and complete a factory repair for a customer's product that is **out of warranty** and has **no serial number**, where the repair result is **Tested OK**. The process starts with a Helpdesk ticket at the receiving branch, moves the item to the factory, records the repair through a Field Service task, and ends when the item is dispatched back to the customer.

Ticket stages covered

New | Sent to Factory | Received at Factory | Diagnosis | Estimation Sent to Customer | Estimation Approval Received | Advance Received | Repair Started | Repair Completed | Sent to Sales Centre | Received at Sales Centre | Handed Over to Customer

Before you start

- You have access to the Helpdesk, Field Service, Inventory, and Accounting apps.
- The active company (top bar) is **Jinasena Agricultural Machinery (Pvt) Ltd.**

- The customer and product already exist in Odoo (Contacts and Product Master).
- The item has been physically brought to the branch by the customer.

Example data used in this manual

Field	Value
Type	Repair - Not Under Warranty (Without Serial No.)
Return Receipt Location	BR-AM/Stock (Jinasena Ampara Branch)
Repair Reason	Difficult to start the engine
Job Location	Factory Repair
Customer	KURUNEGALA CASH CUSTOMER
Product	[JMC6] Multi Chopper 6Hp (Tractor Type)
Spare Part	[02BB 023] BALL BEARING 6202-2RS
Payment Journal	Commercial Bank
Repair result	Tested OK

How to read each step

Navigation	Where you are in Odoo (app › menu › screen / dialog).
Action	What to click, select, or type.
Description	Why the step is needed and what happens.
Screenshot	The screen you're on *right before* performing the action, with the element to use framed in red — from a live Odoo instance, captured by a single clean, fully-passing automated run. A few steps could not be captured this way (hidden actions driven directly against the server, or moments the automation didn't screenshot separately) and are noted as such.

Step summary

Section	Steps	Purpose
A	1–11	Open Helpdesk, create the ticket, and classify it (type, locations, reason, customer, product) through to generating a temporary serial number.

B	12–14	Validate the item's receipt into stock and move it on to the factory.
C	15–17	Create the Field Service task, record the diagnosis, and attach a repair photo.
D	18–22	Add spare parts, generate the quotation, send it, and confirm it.
E	23–27	Invoice and collect a 50% deposit, then validate the delivery chain it unlocks.
F	28–33	Close the task, route the item through the sales centre, collect the balance, and dispatch it back to the customer.

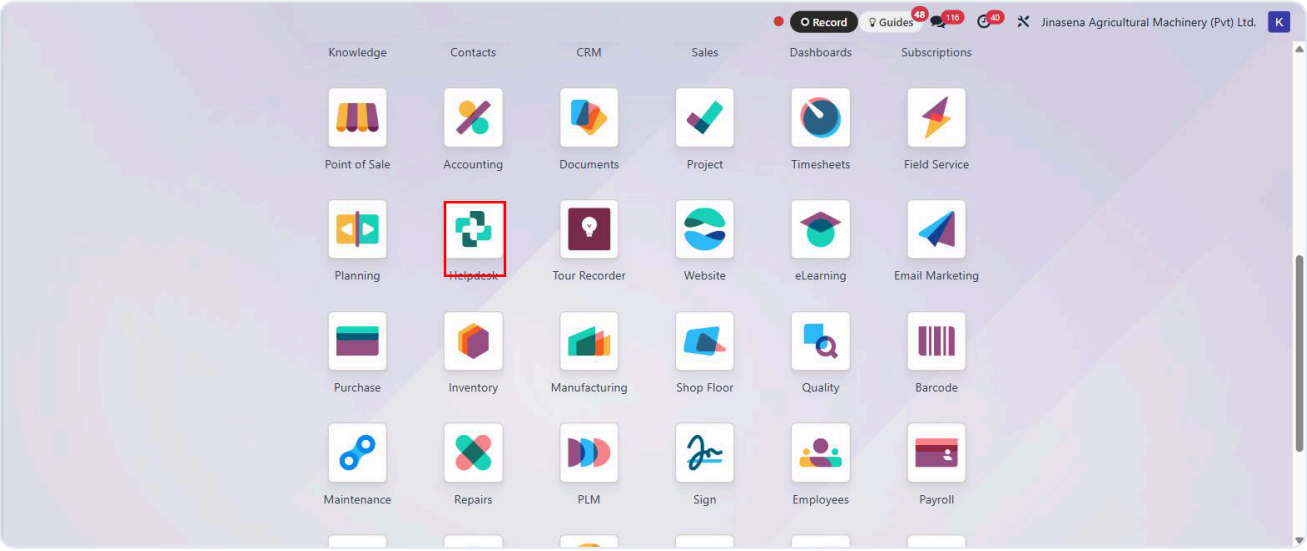
Section A — Create the Repair Ticket (Steps 1–11)

1 Open the HelpDesk app

Navigation Odoo Home → Apps

Action **Click the Helpdesk app icon on the home screen.**

Description The Odoo home screen lists all installed apps as icons. Clicking Helpdesk opens its Overview dashboard, showing every support team you have access to.



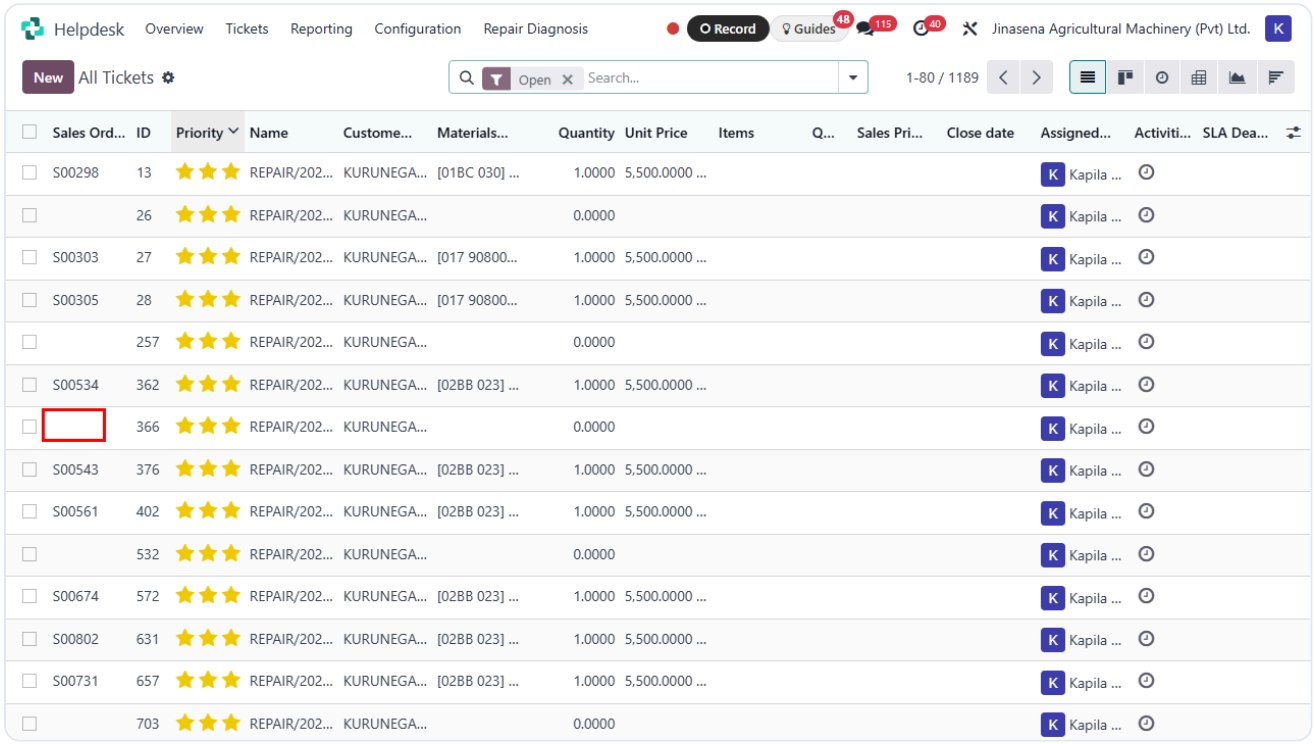
Odoo Apps home screen — Helpdesk app icon

2 Open the repair tickets

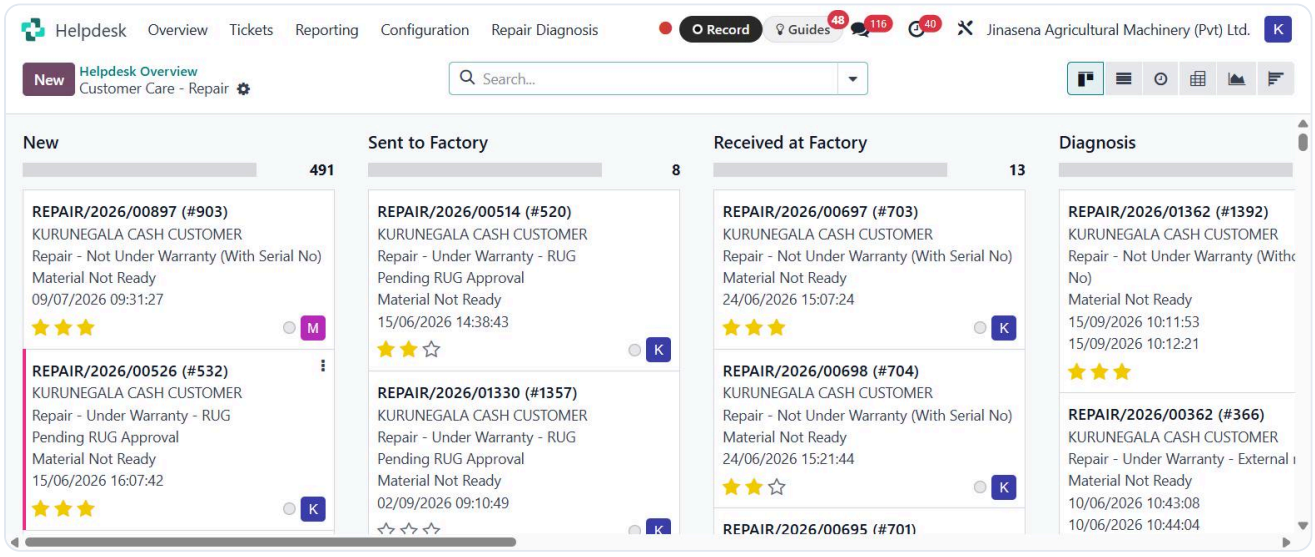
Navigation Helpdesk → Overview → Customer Care - Repair

Action **On the Customer Care - Repair team card, click Tickets.**

Description The Helpdesk Overview shows each helpdesk team as a card with ticket counts. The Tickets button opens the list of repair tickets already created for that team, arranged by stage.



Helpdesk Overview — Tickets button on the Customer Care - Repair team card



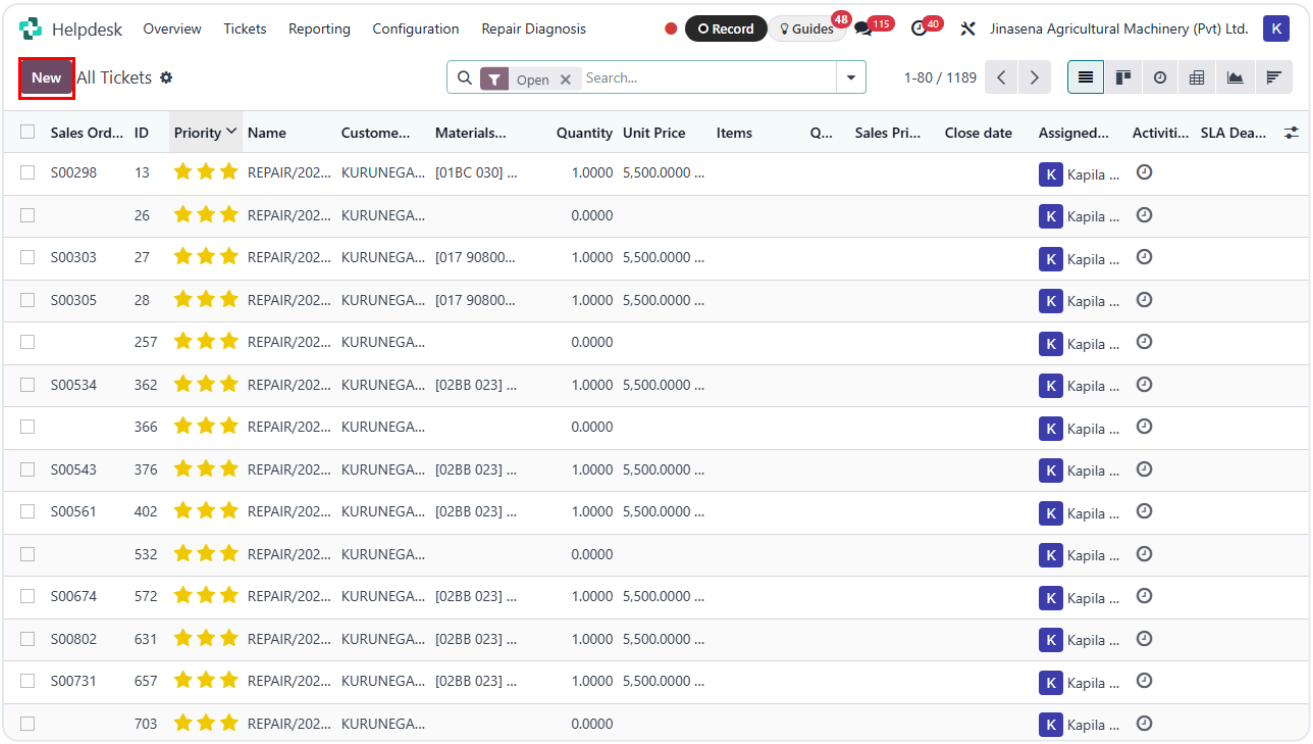
Ticket Kanban board, opened after clicking Tickets

3 Create a new ticket

Navigation Helpdesk → Customer Care - Repair (Tickets)

Action **Click New at the top-left of the ticket board.**

Description Opens a blank Helpdesk ticket form for the repair item received from the customer.



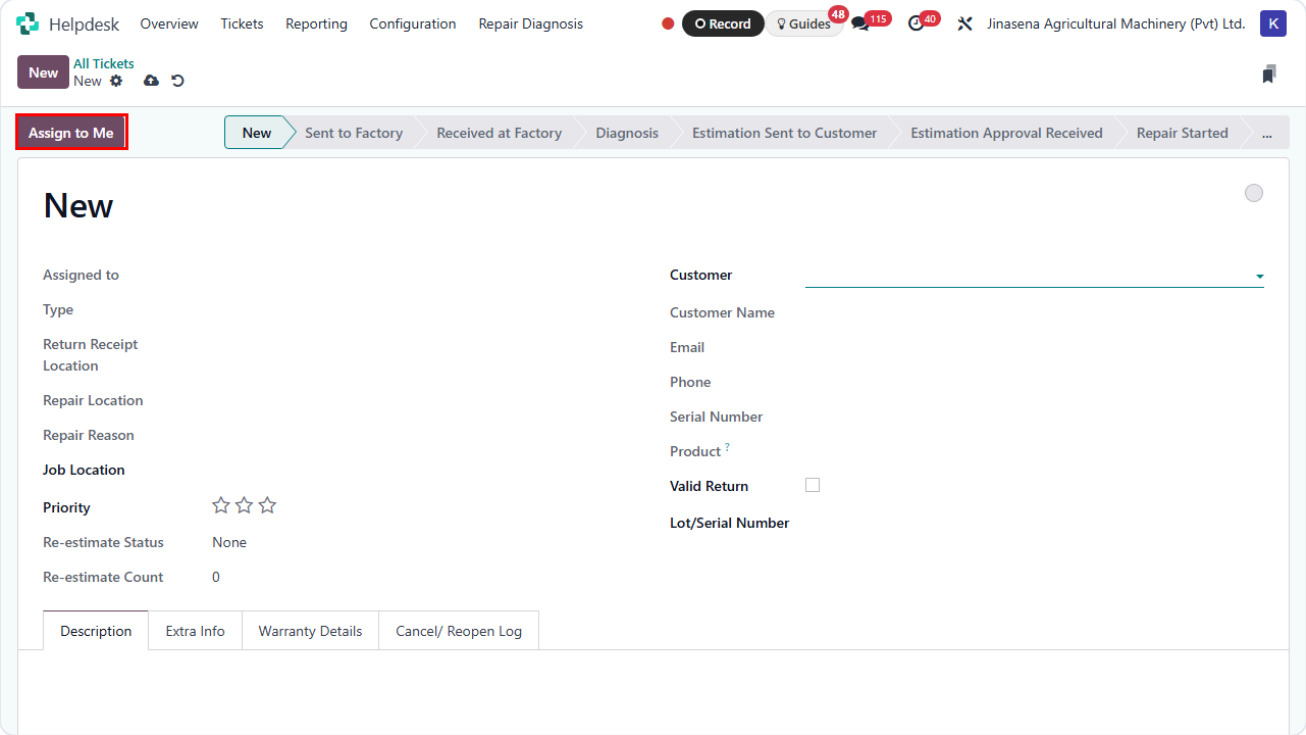
All Tickets list — New button

4 Assign the ticket to yourself

Navigation New ticket form

Action **Click Assign to Me in the status bar.**

Description Saves the ticket for the first time and assigns it to the current user. This also triggers the ticket's sequence-based name (e.g. `REPAIR/2026/01524`) to be generated, since the ticket has no user-editable title field.



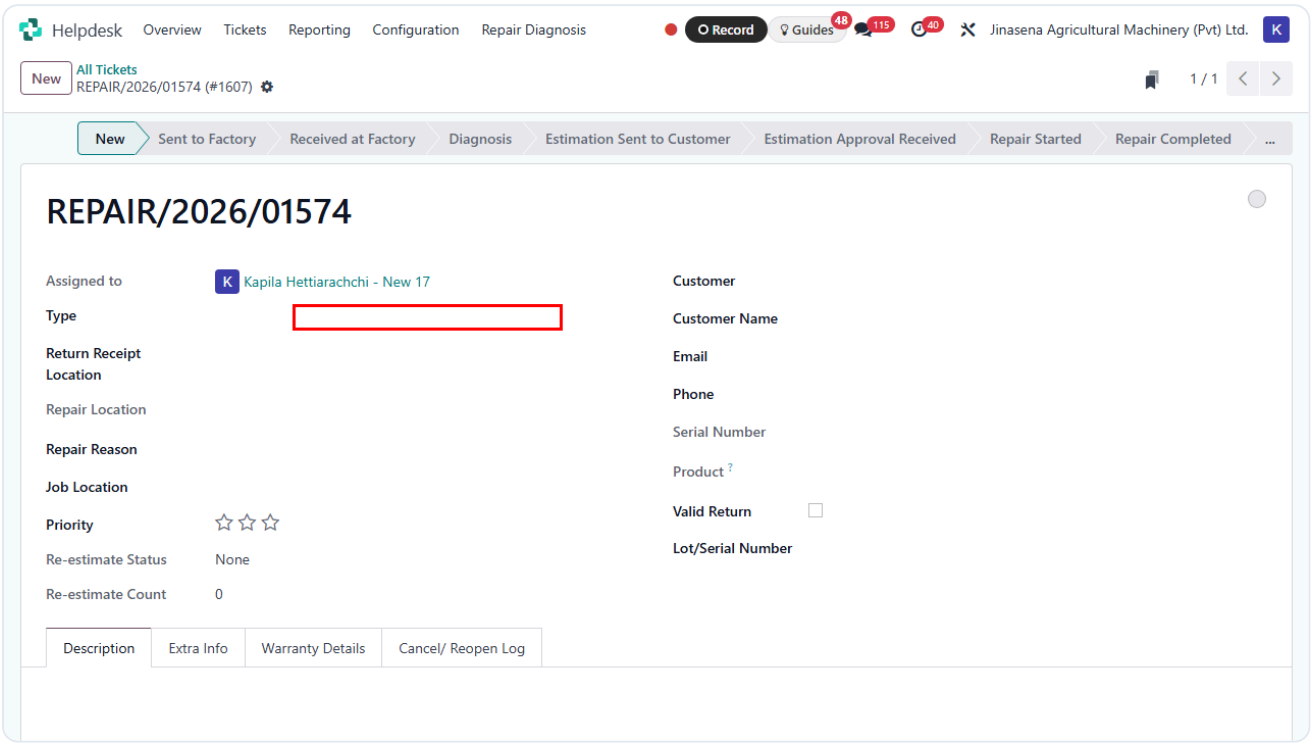
Blank new ticket form — Assign to Me button

5 Set the repair Type

Navigation Ticket form → Type field

Action **Click the Type field and select Repair - Not Under Warranty (Without Serial No.) from the list.**

Description This field is a many2one autocomplete (not a plain dropdown) listing the repair types configured in Studio. This type classifies items whose warranty has expired and which have no serial number.



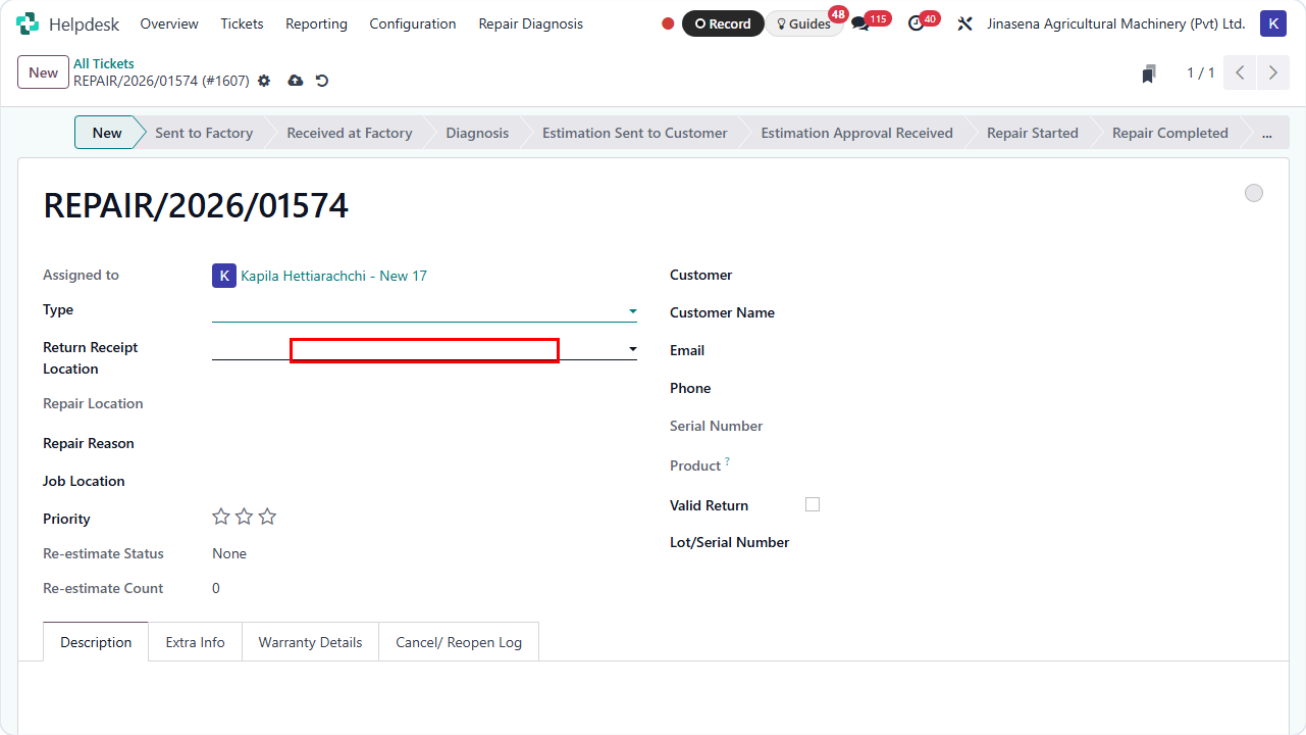
Ticket after Assign to Me — Type field ready to be set

6 Set the Return Receipt Location

Navigation Ticket form → Return Receipt Location field

Action **Click the Return Receipt Location field and select the branch receiving the item (e.g. `BR-AM/Stock` — Jinasena Ampara Branch).**

Description Marks which branch stock location the item is physically received into before being sent to the factory.



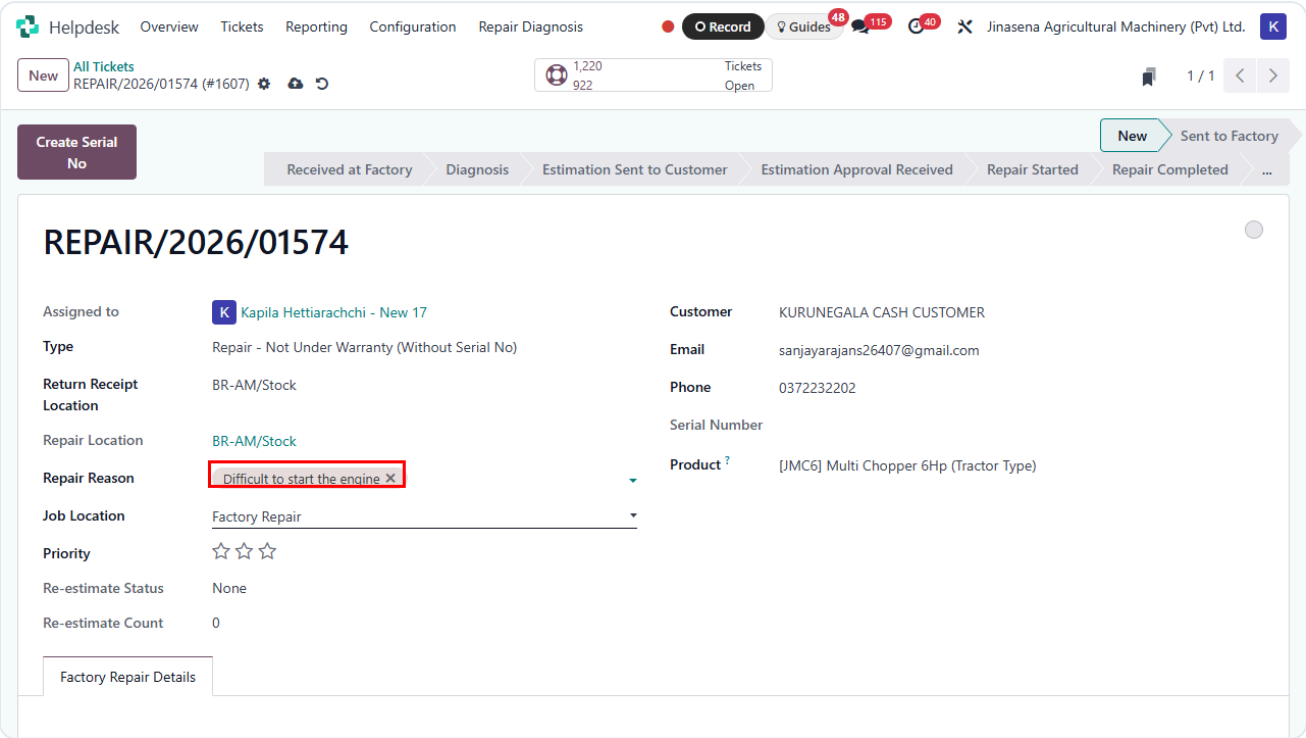
Type set — Return Receipt Location field ready to be set

7 Set the Repair Reason

Navigation Ticket form → Repair Reason field

Action **Click the Repair Reason field, type to search, and select the closest matching common error tag (e.g. "Difficult to start the engine").**

Description A many2many tags field describing the issue the customer reported. Required before the ticket can proceed to Create Serial No.



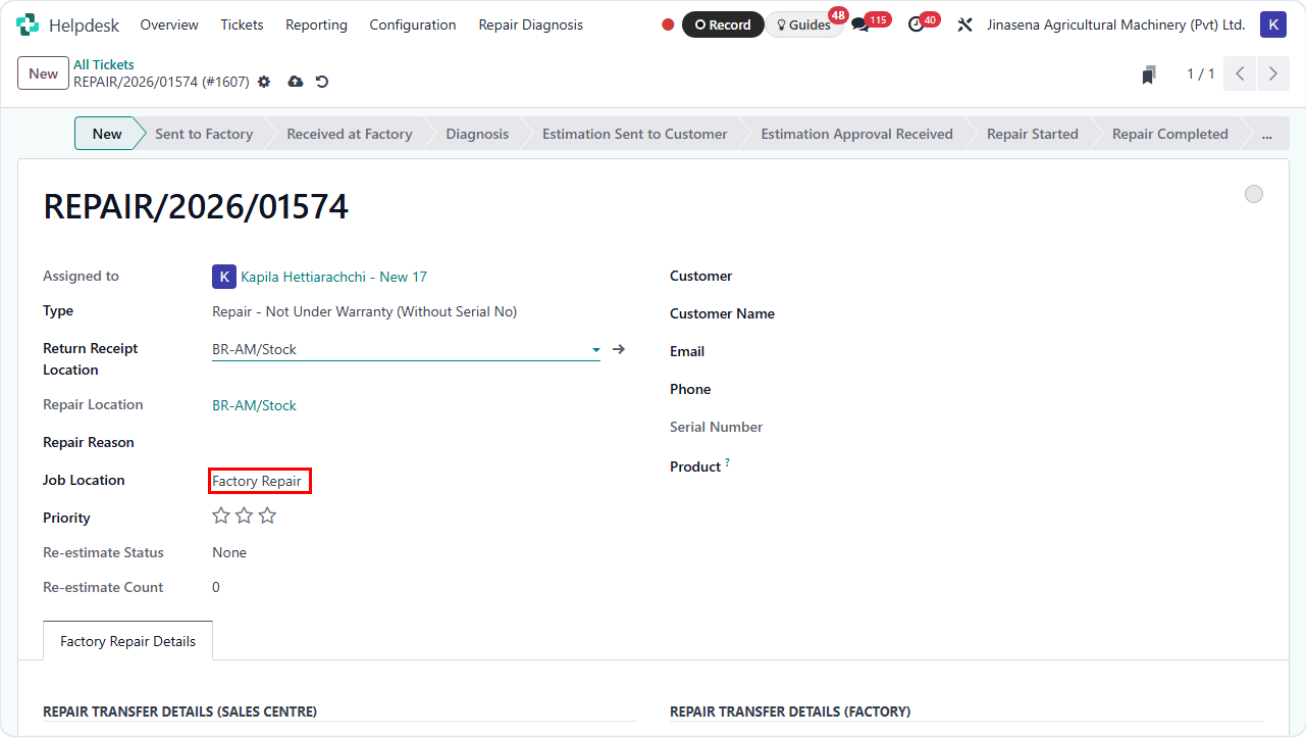
Repair Reason tag set to Difficult to start the engine

8 Set the Job Location

Navigation Ticket form → Job Location field

Action **Click the Job Location dropdown and select Factory Repair.**

Description A plain HTML select field marking whether the repair is done locally at the branch or sent to the factory.



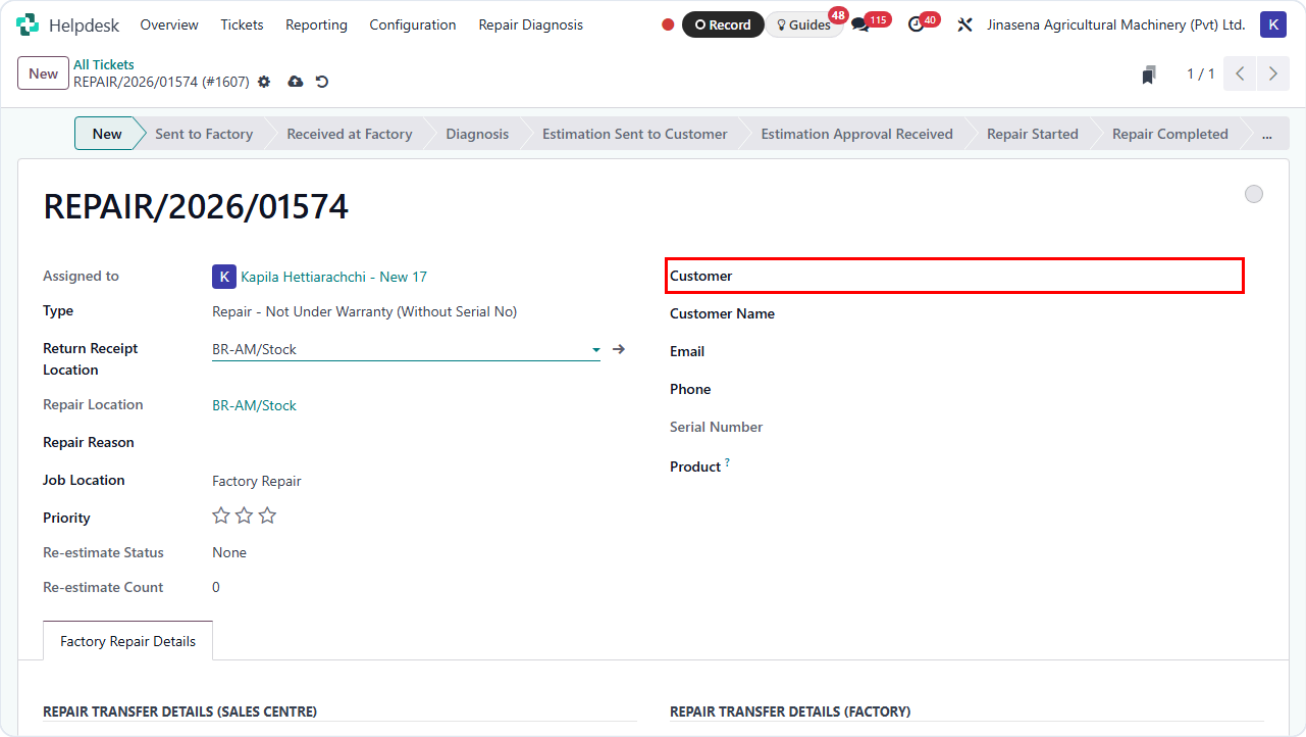
Job Location set to Factory Repair

9 Select the Customer

Navigation Ticket form → Customer field

Action **Click the Customer field and select the customer's contact (e.g. `KURUNEGALA CASH CUSTOMER`).**

Description Links the ticket to the customer's existing contact record in Odoo.



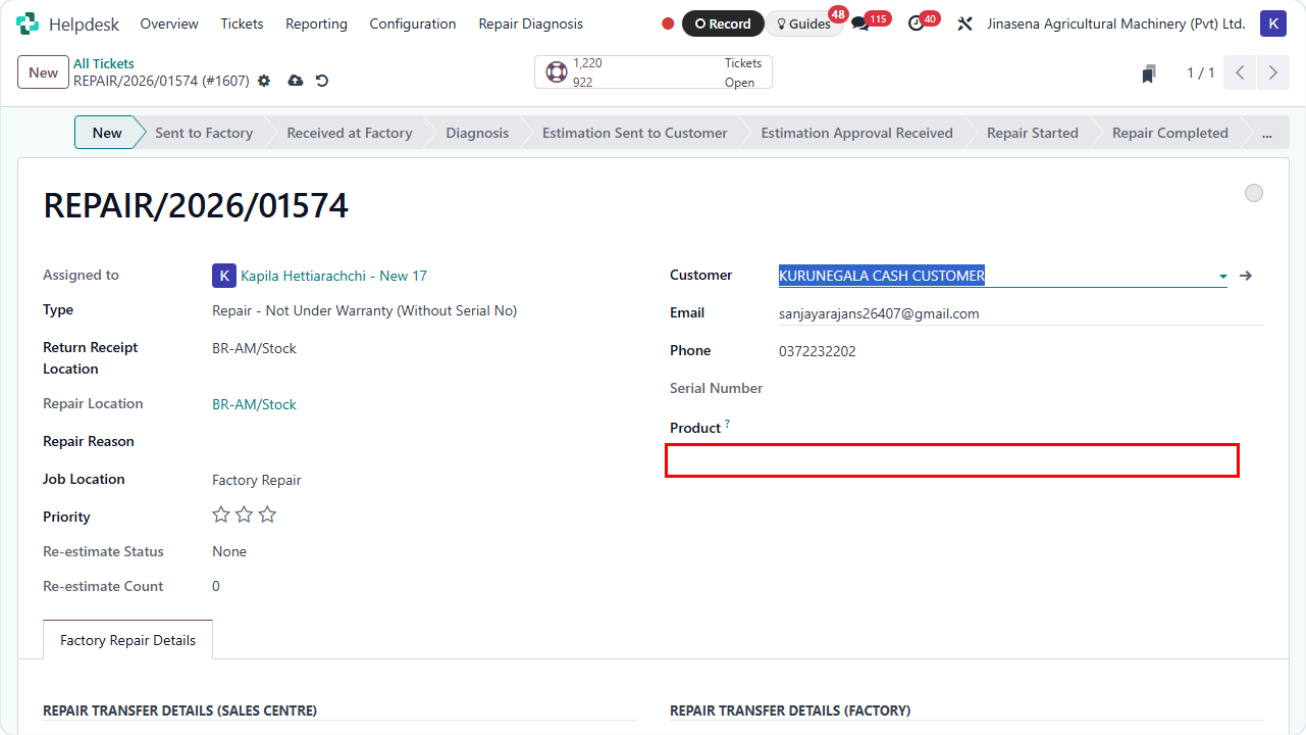
Customer field set to KURUNEGALA CASH CUSTOMER

10 Select the Product

Navigation Ticket form → Product field

Action **Click the Product field, search by the product code, and select the exact product being repaired (e.g. `[JMC6] Multi Chopper 6Hp (Tractor Type)`).**

Description Links the ticket to the product master record. Searching by the short product code is more reliable than the full bracketed display name.



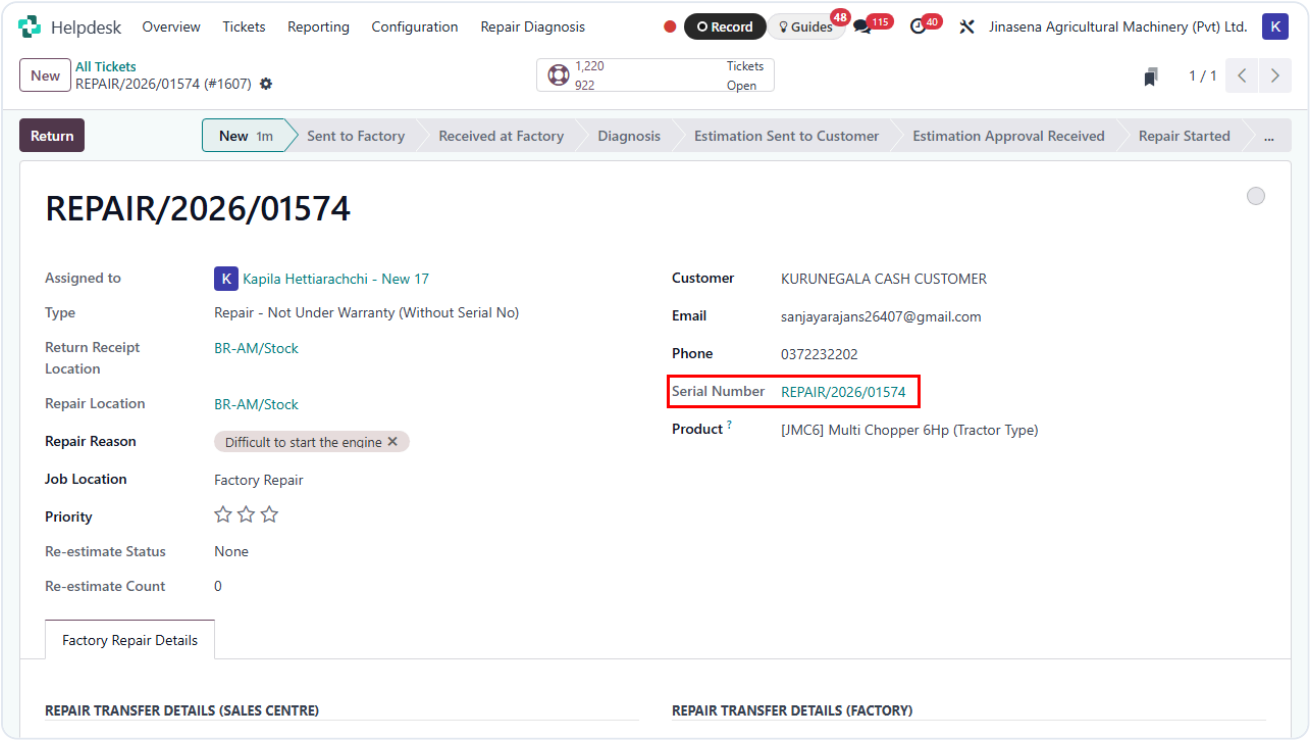
Product field set to [JMC6] Multi Chopper 6Hp (Tractor Type)

11 Create a temporary serial number

Navigation Ticket form status bar

Action **Click Create Serial No.**

Description Because this repair type has no serial number, Odoo generates a temporary one from the ticket's own sequence name. The ticket must be fully saved (no pending changes) for this button to take effect — this requires Repair Reason (step 7) to already have a value.



Serial Number generated, Return button now available

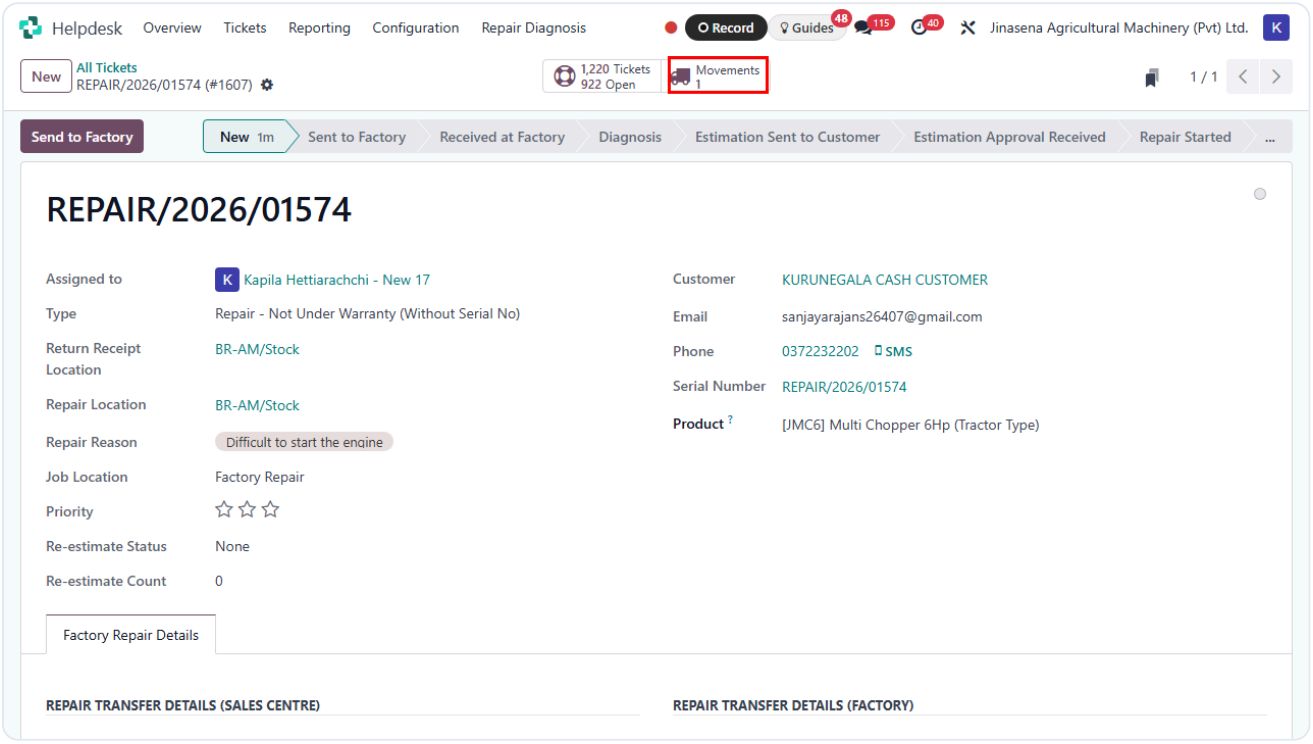
Section B — Receive and Send to Factory (Steps 12–14)

12 Validate the return receipt

Navigation Ticket form (driven directly)

Action **Confirmed live: the status-bar Return button always fails server-side with a real bug in the Fix-Repair customization. The fix calls the same underlying method directly, then validates the "Returned Picking" it creates.**

Description A "Returned Picking" transfer (e.g. `BR-AM/RET/...`) records the item being physically received from the customer into company stock — this is a hard prerequisite for "Send to Factory" to appear.



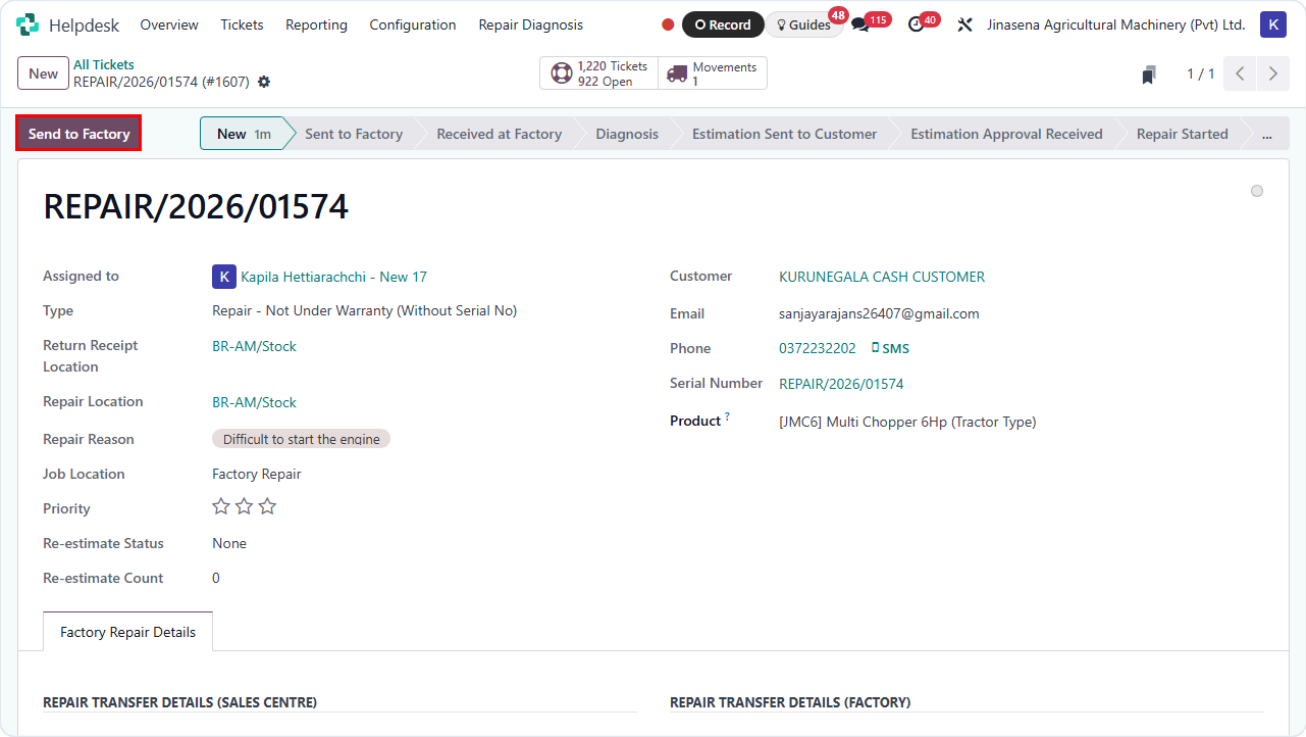
Ticket with the return picking validated — Movements stat button confirms it exists

13 Send to Factory

Navigation Ticket form status bar

Action **Click Send to Factory.**

Description Advances the ticket's stage and moves the item onward for repair work at the factory. Only appears once the return picking above is validated.



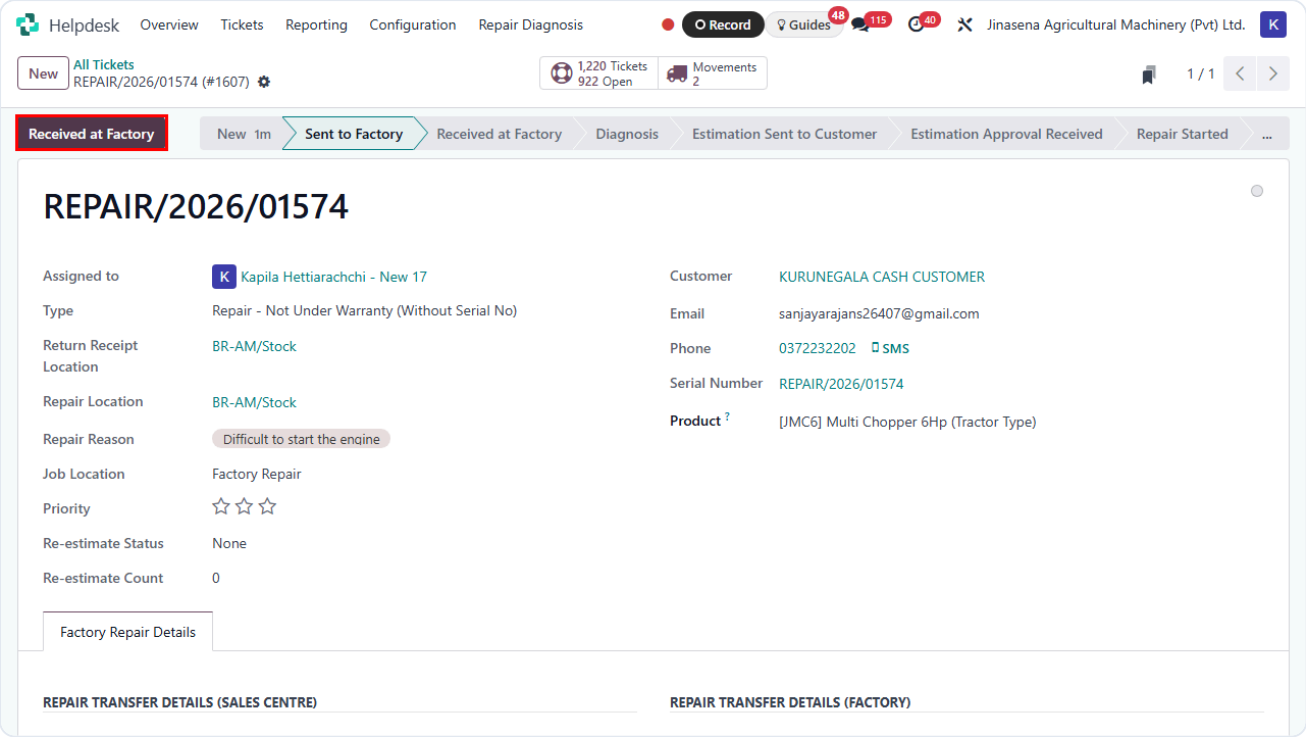
Send to Factory button ready after the return picking was validated

14 Confirm receipt at the factory

Navigation Ticket form status bar

Action **Click Received at Factory.**

Description Confirms the item has physically arrived at the factory and is ready for a technician to be assigned.



Ticket at Sent to Factory stage — Received at Factory button

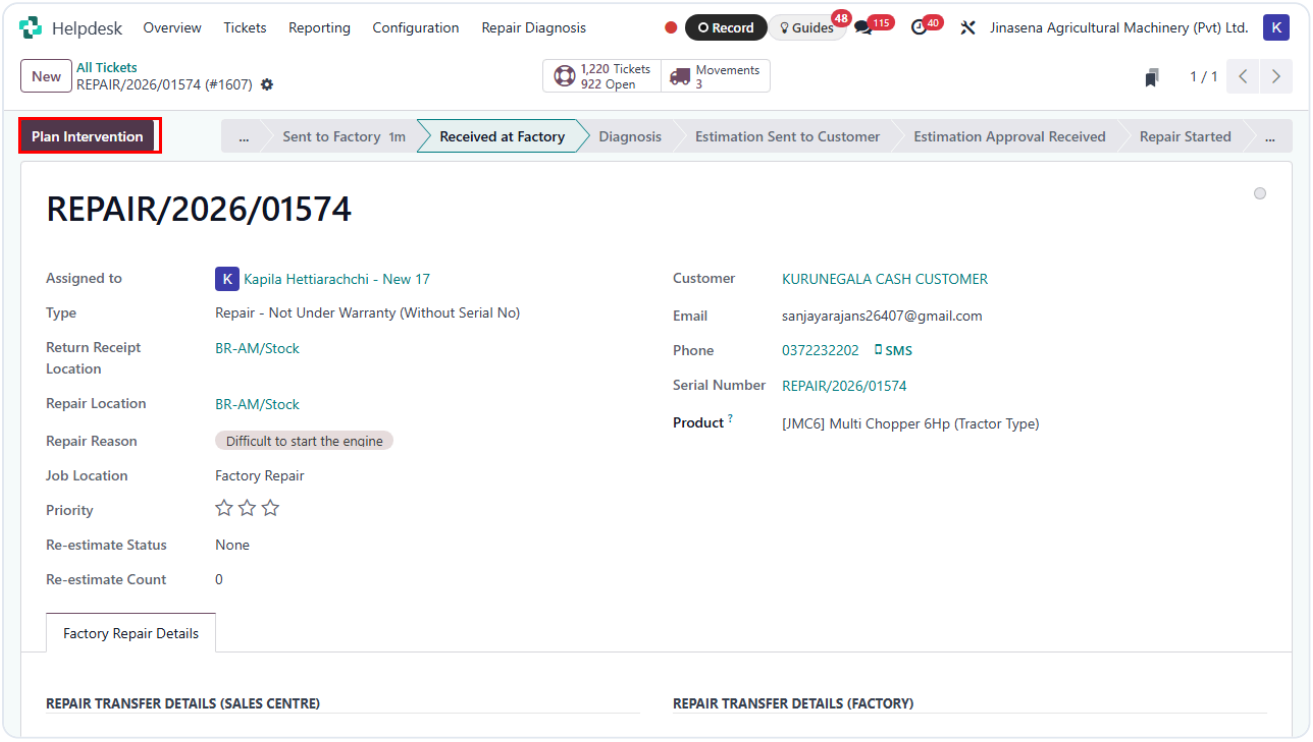
Section C — Plan the Repair Task (Steps 15–17)

15 Plan the intervention and open the task

Navigation Ticket form status bar

Action **Click Plan Intervention, then click Create & View Task in the dialog that appears.**

Description Plan Intervention assigns a technician; Create & View Task then opens the Field Service task created for the repair, in a dialog on top of the ticket.



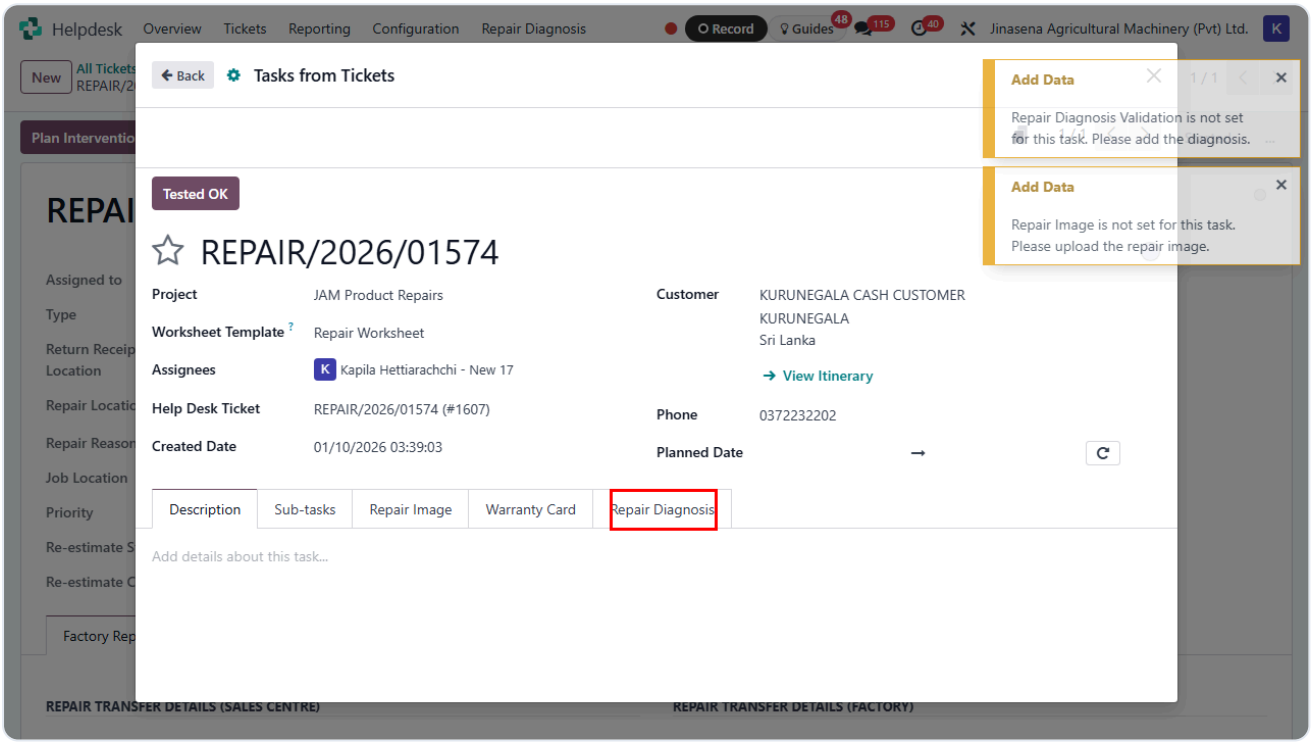
Ticket at Received at Factory stage — Plan Intervention button

16 Record the Repair Diagnosis

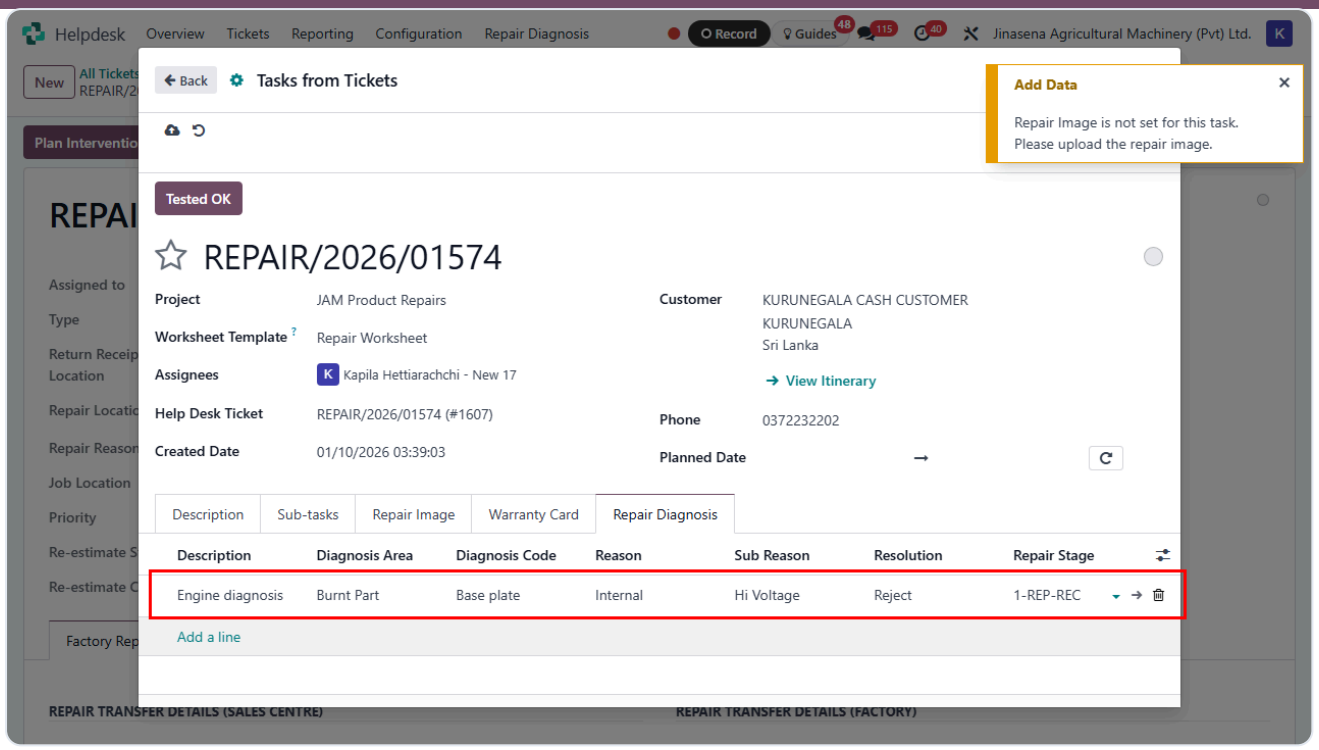
Navigation Task form → Repair Diagnosis tab

Action **Click Add a line, then fill in Diagnosis Area, Diagnosis Code, Reason, Sub Reason, and Resolution/Repair Stage from the reference dropdowns.**

Description Records the technician's findings using the diagnosis reference tables (example values used: Burnt Part / Base plate / Internal / Hi Voltage / Reject / 1-REP-REC).



Field Service task — Repair Diagnosis tab



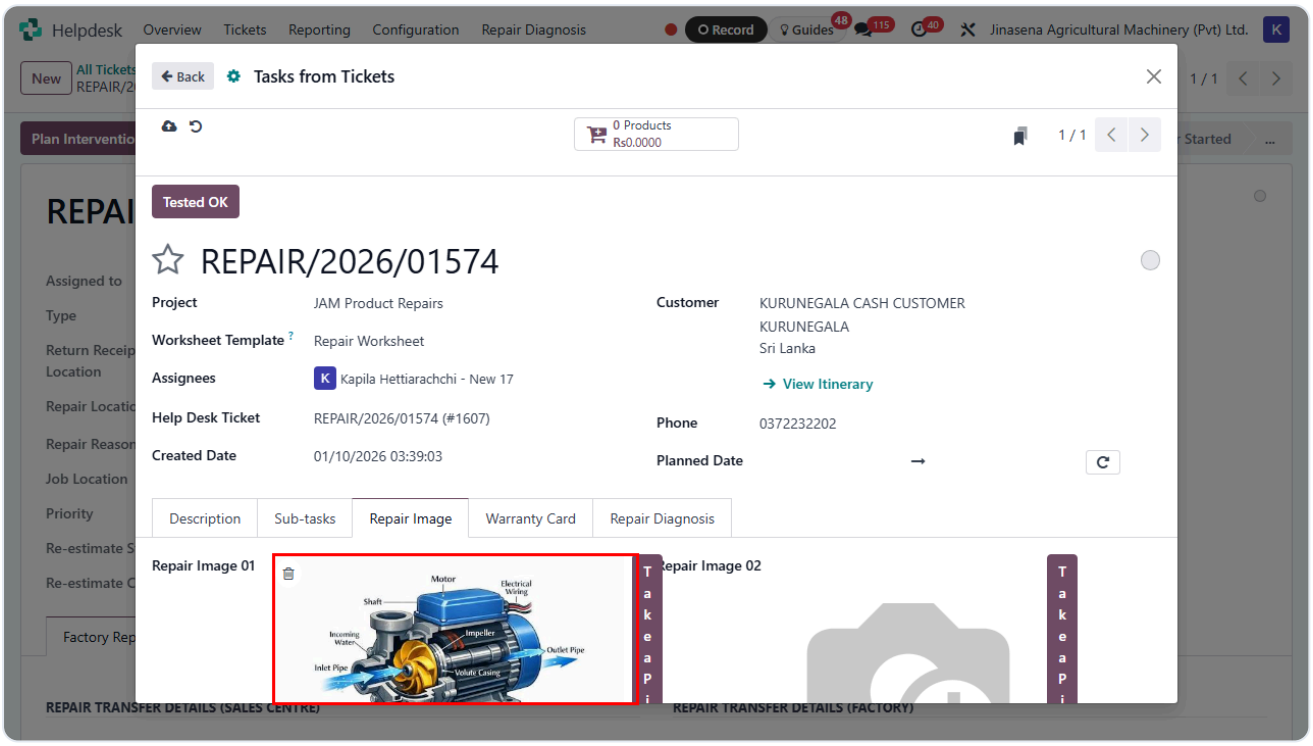
Repair Diagnosis line recorded

17 Attach the Repair Image

Navigation Task form → Repair Image tab

Action **Click the Repair Image 01 upload area and choose a photo file of the item.**

Description The task template provides two separate image slots; at least one photo is required before the Products button will appear.



Repair photo attached to the Repair Image 01 slot

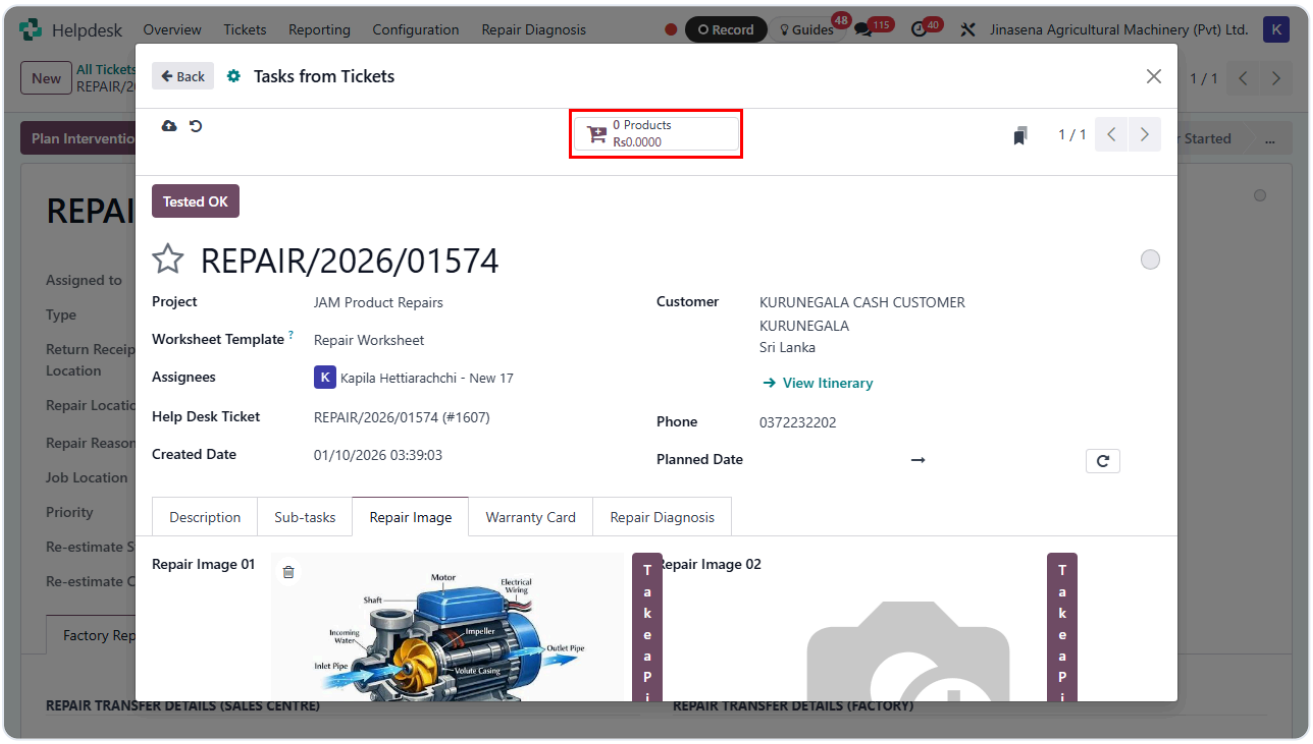
Section D — Quote and Confirm Parts (Steps 18–22)

18 Add spare parts

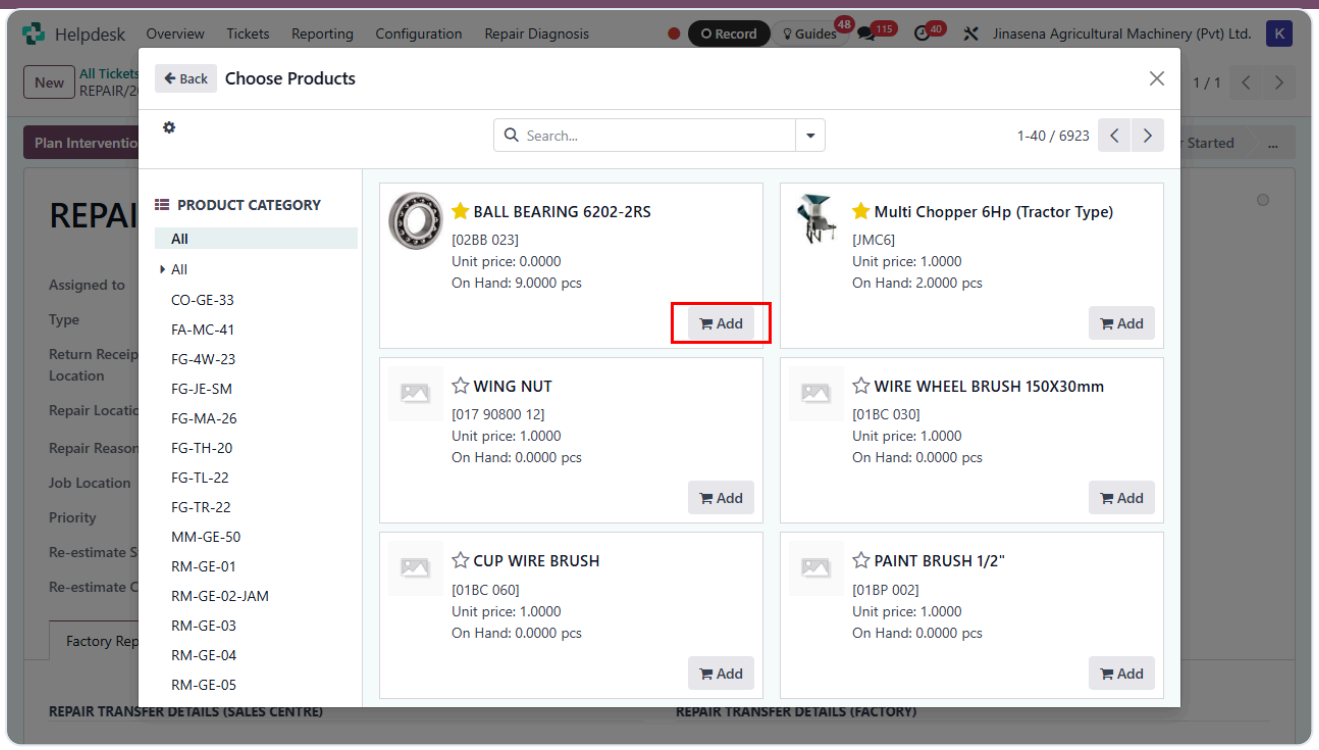
Navigation Task form header

Action **Click the Products stat button, then use the catalog's search box to find the part and click Add.**

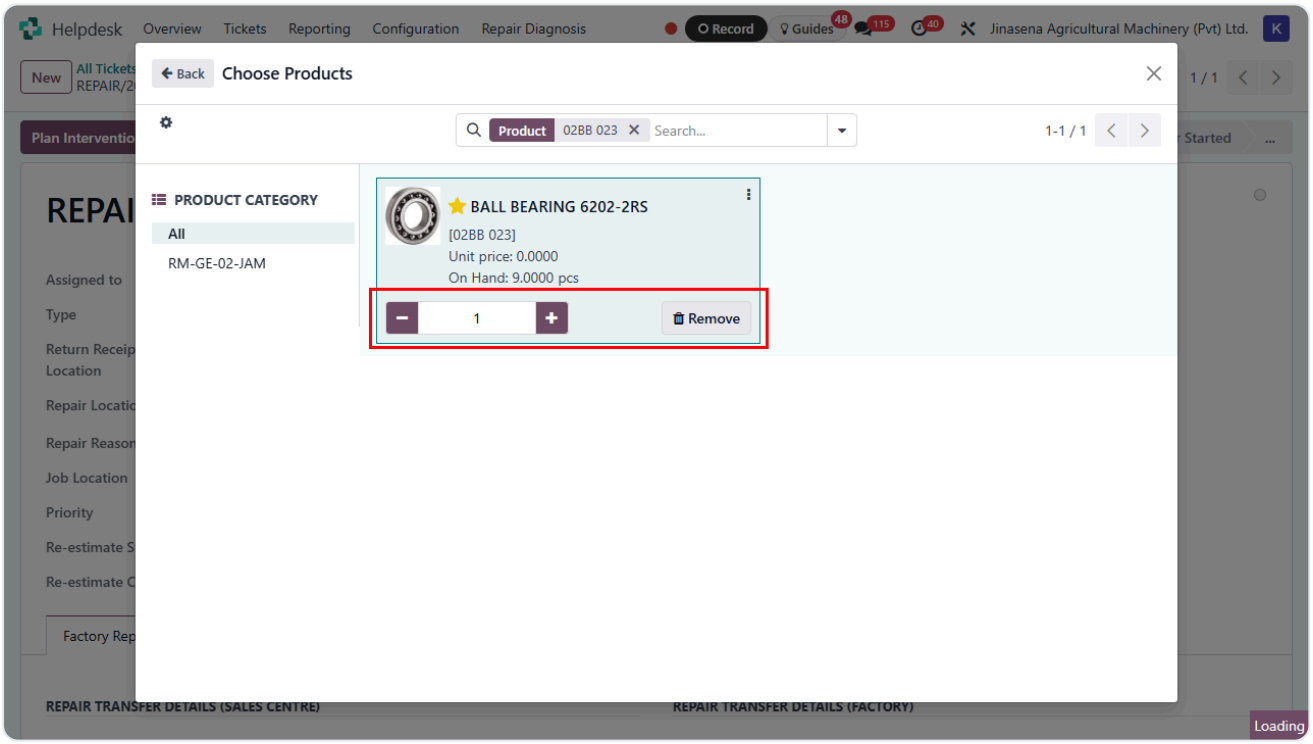
Description This button only appears once Repair Diagnosis and Repair Image are both set. Example part: `[02BB 023] BALL BEARING 6202-2RS`, found by searching its product code rather than scanning the catalog visually.



Task header — 0 Products stat button



Choose Products catalog — Add button



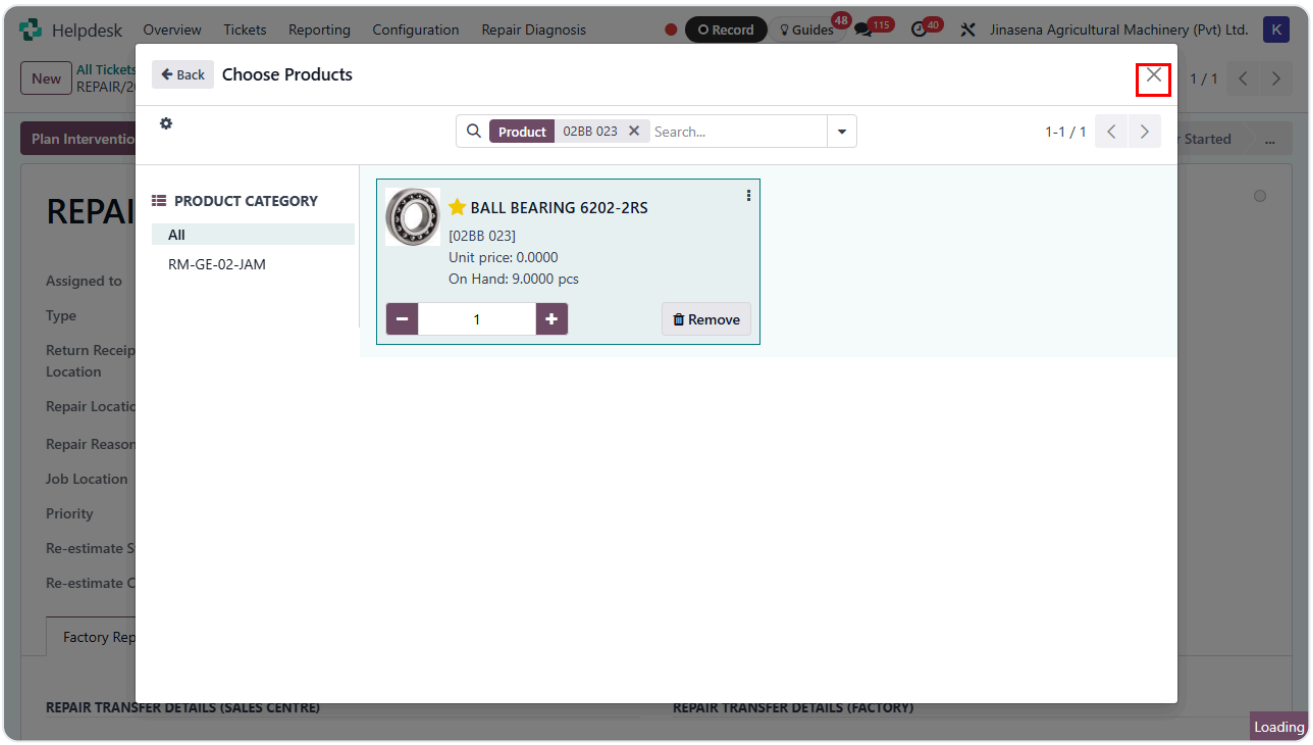
Part added with quantity 1

19 Close the product catalog

Navigation Choose Products catalog

Action **Click the catalog's close icon (×).**

Description The ticket's status bar advances automatically to "Diagnosis" once a part is added; closing the catalog lands back on the ticket rather than the task.



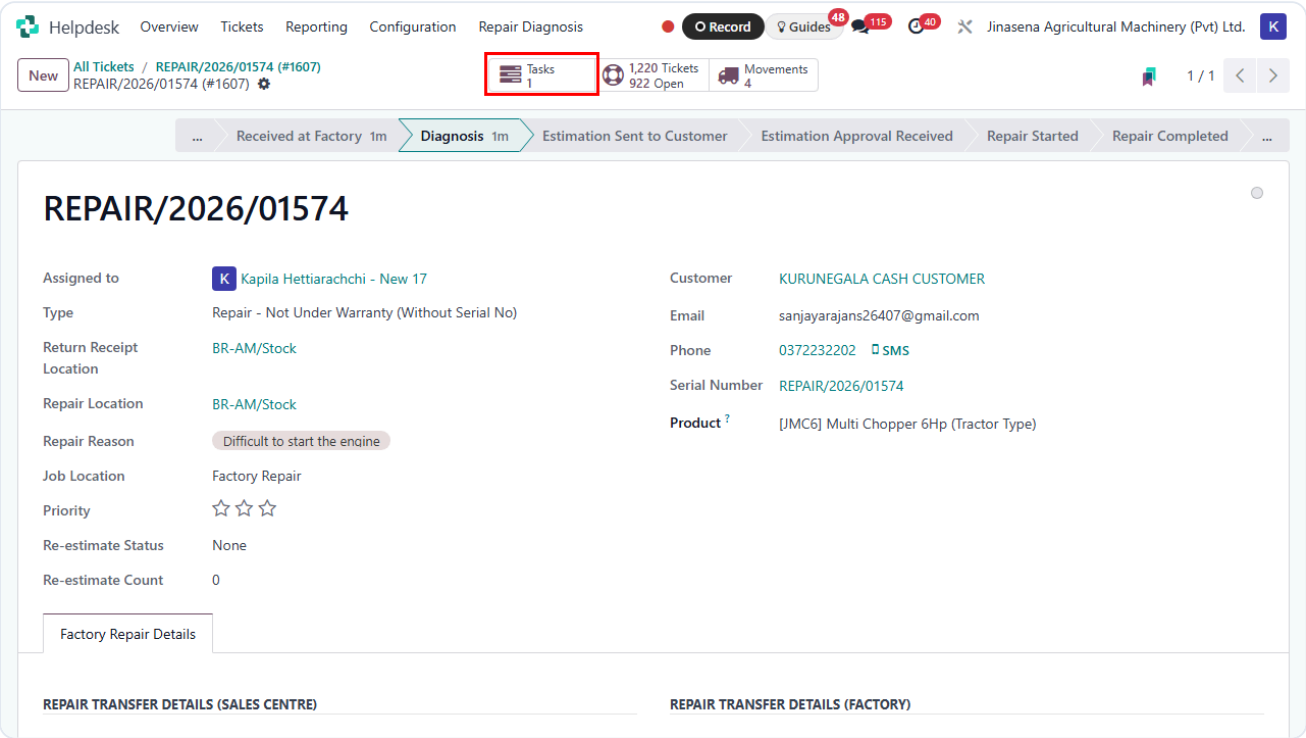
Choose Products catalog — close icon

20 Open the generated quotation

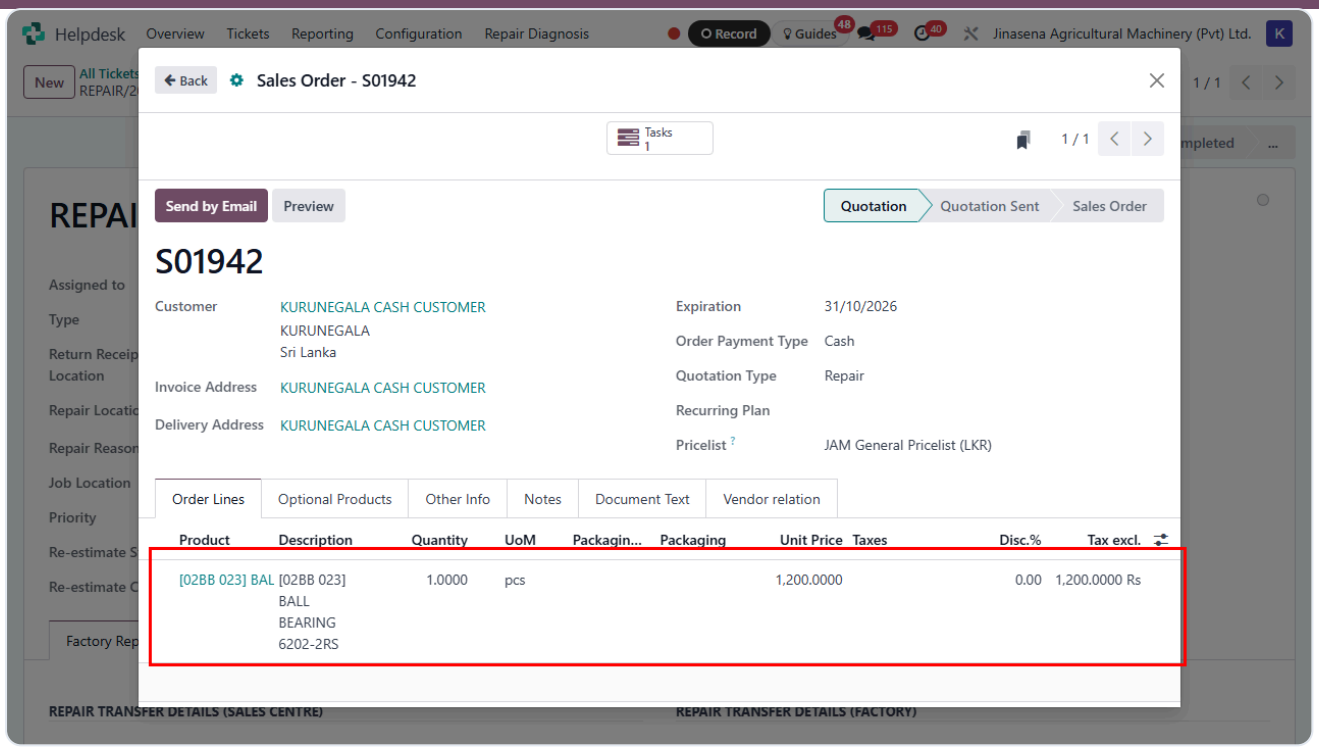
Navigation Ticket form header

Action **Click the Tasks stat button to reopen the task, then click the Sales Order stat button that now appears there.**

Description The Sales Order stat button only renders once the task has priced order lines. Clicking it opens the quotation Odoo generated from the parts added.



Ticket — Tasks stat button



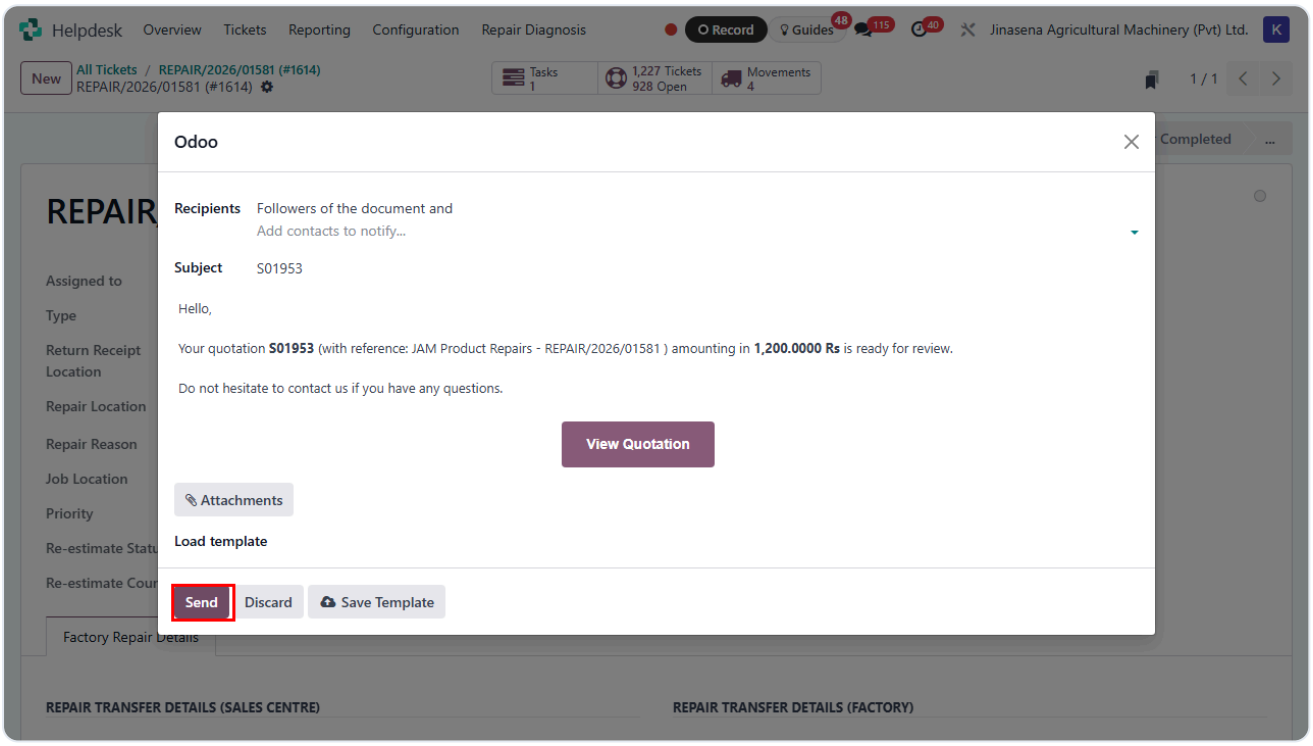
Sales Order quotation opened, showing the part line

21 Send the quotation

Navigation Sales Order form

Action **Click Send by Email, then click Send in the dialog.**

Description Emails the quotation to the customer for approval. The email includes a "View Quotation" button linking to the customer-facing portal page.



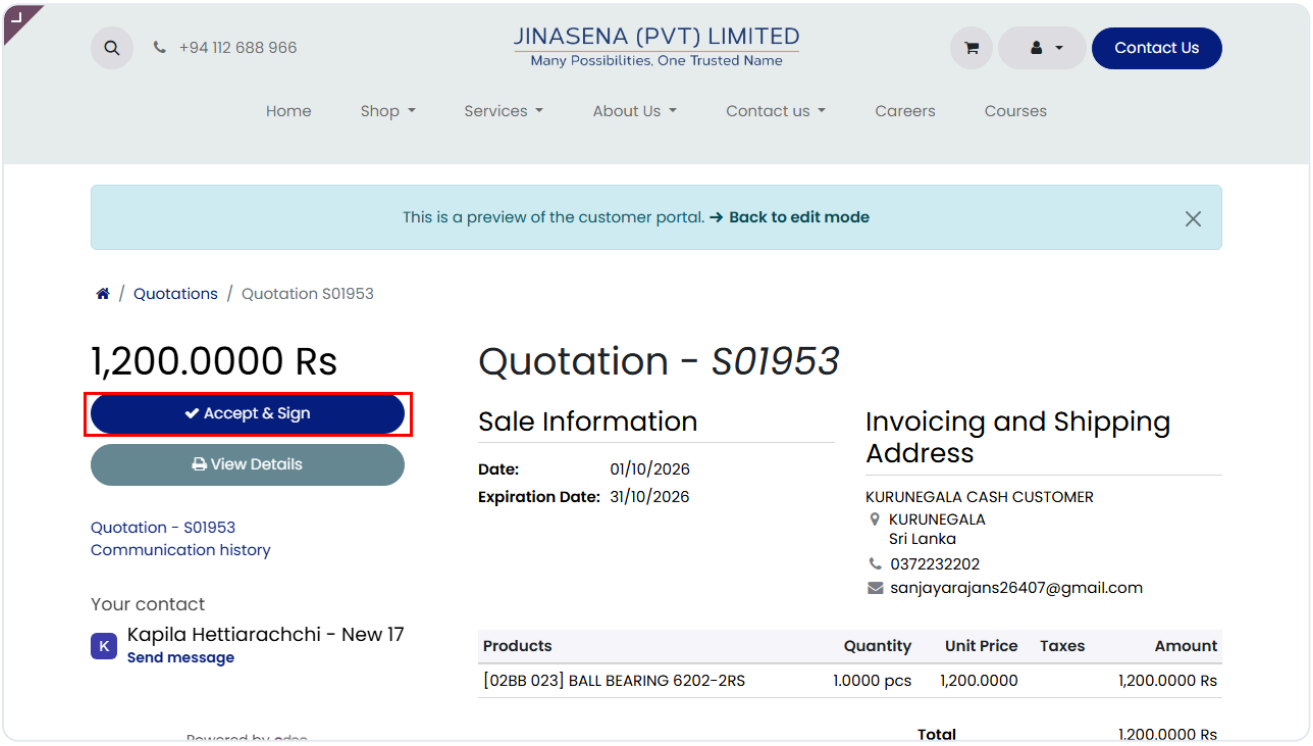
Email composer confirming the quotation — Send boxed

22 Confirm the Sales Order

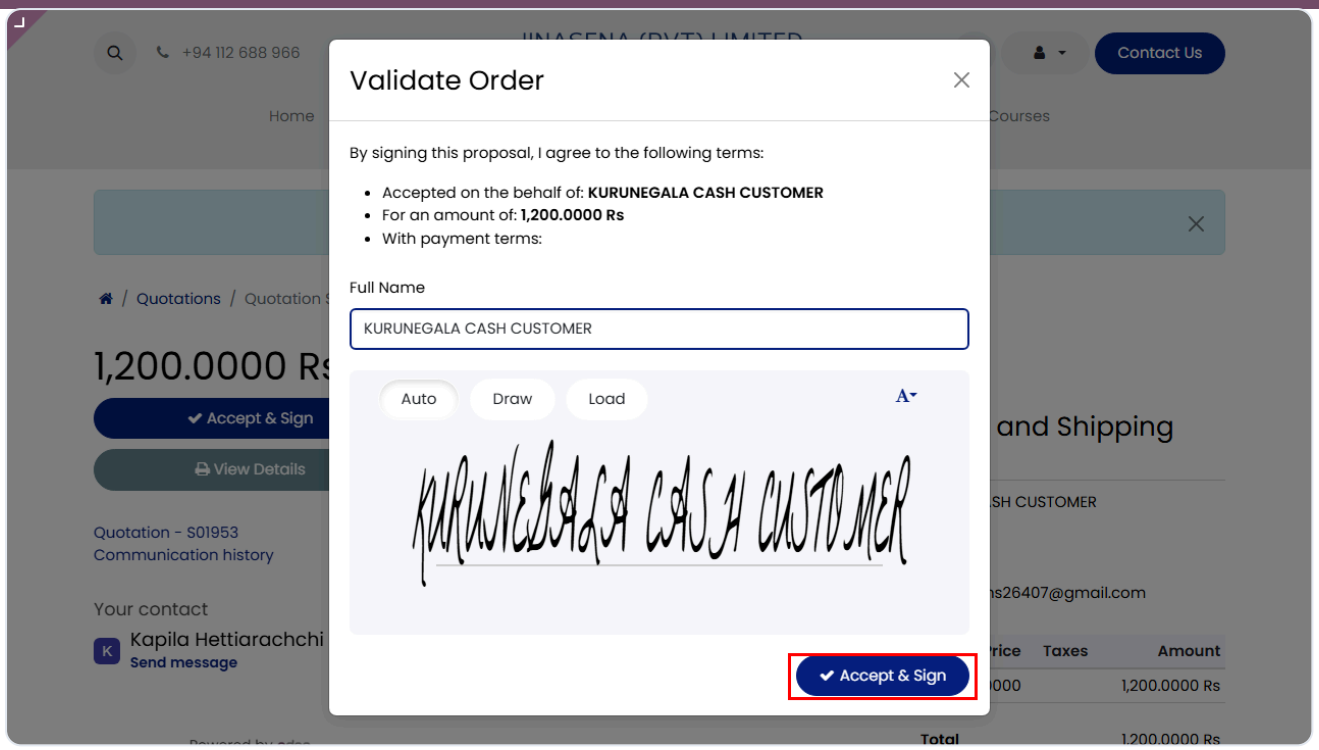
Navigation Sales Order form → customer portal (public)

Action **Open the "View Quotation" link from the email, then click Accept & Sign on the portal page, and click Accept & Sign again on the signature popup.**

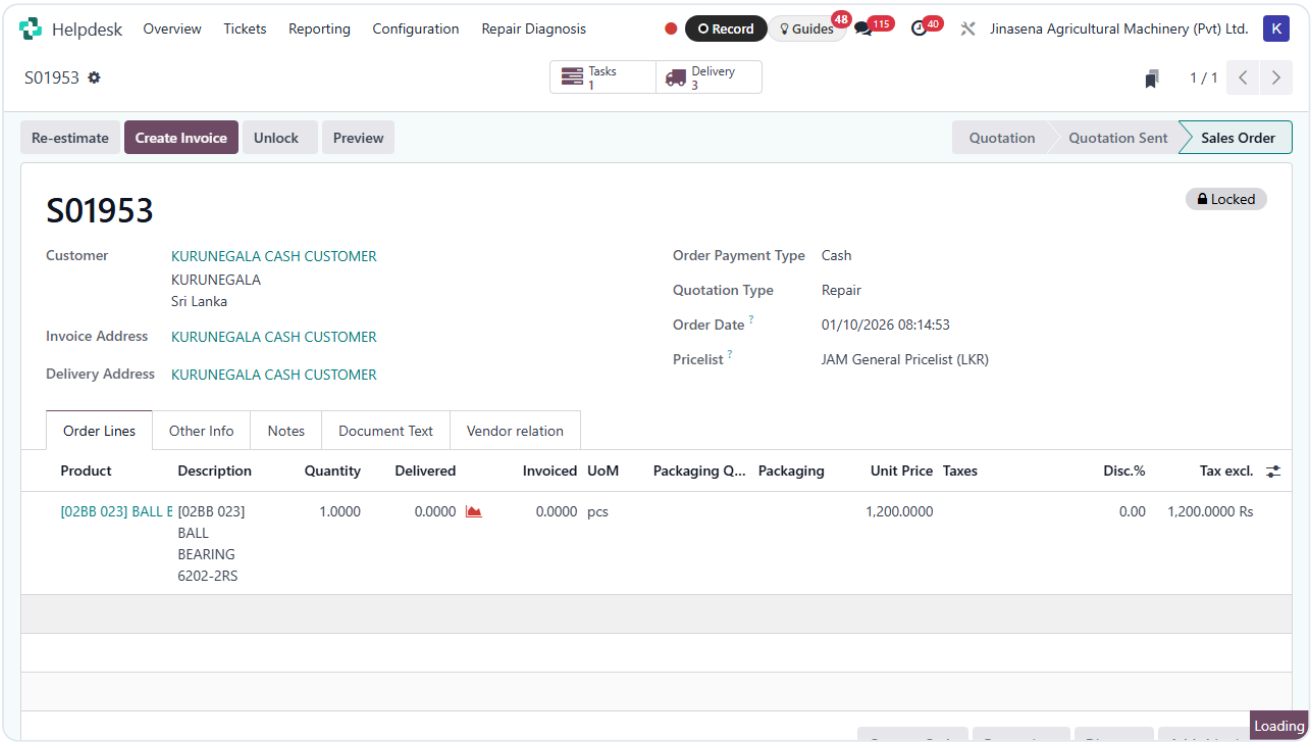
Description This is the real path a customer follows to approve a repair quotation online — no backend access needed. Opening the emailed link lands on the public portal page for the quotation, showing the price and an "Accept & Sign" button. Clicking it opens a "Validate Order" popup with the customer's name and an auto-generated signature already filled in; clicking "Accept & Sign" there submits it. The Sales Order instantly moves from "Quotation Sent" to "Sales Order" and locks, generating a "Delivery 3" stat button since three linked stock moves are created in one go. (On this instance the "Confirm" button is also directly accessible on the order form as a fallback, but the automated flow drives the same online flow a real customer uses.)



Customer-facing portal page for the quotation — Accept & Sign boxed



Validate Order signature popup — Accept & Sign boxed



Sales Order confirmed and locked — Delivery 3 stat button

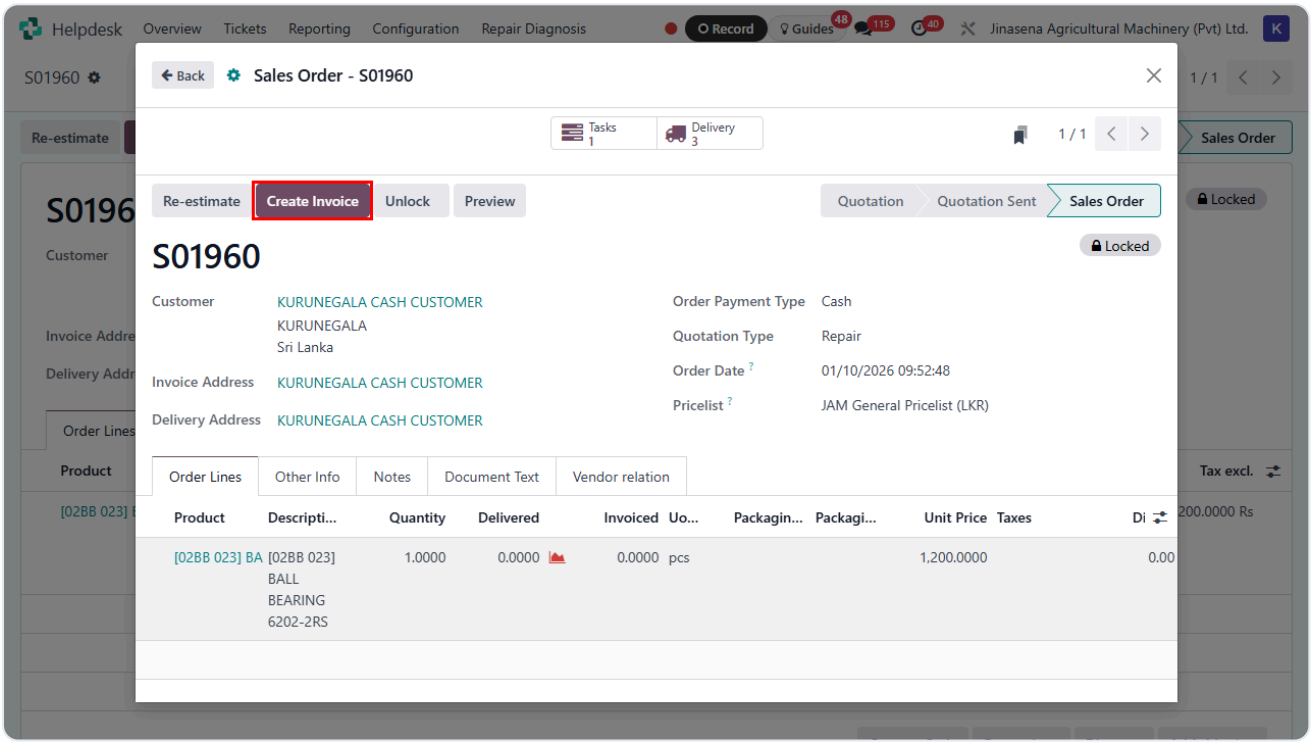
Section E — Deposit and Delivery (Steps 23–27)

23 Create the deposit invoice

Navigation Sales Order form

Action **Click Create Invoice.**

Description Company policy requires a 50% deposit before repair work proceeds. On this instance it creates a full-amount draft invoice directly (no percentage wizard); the deposit itself is taken as a partial payment in the next step.



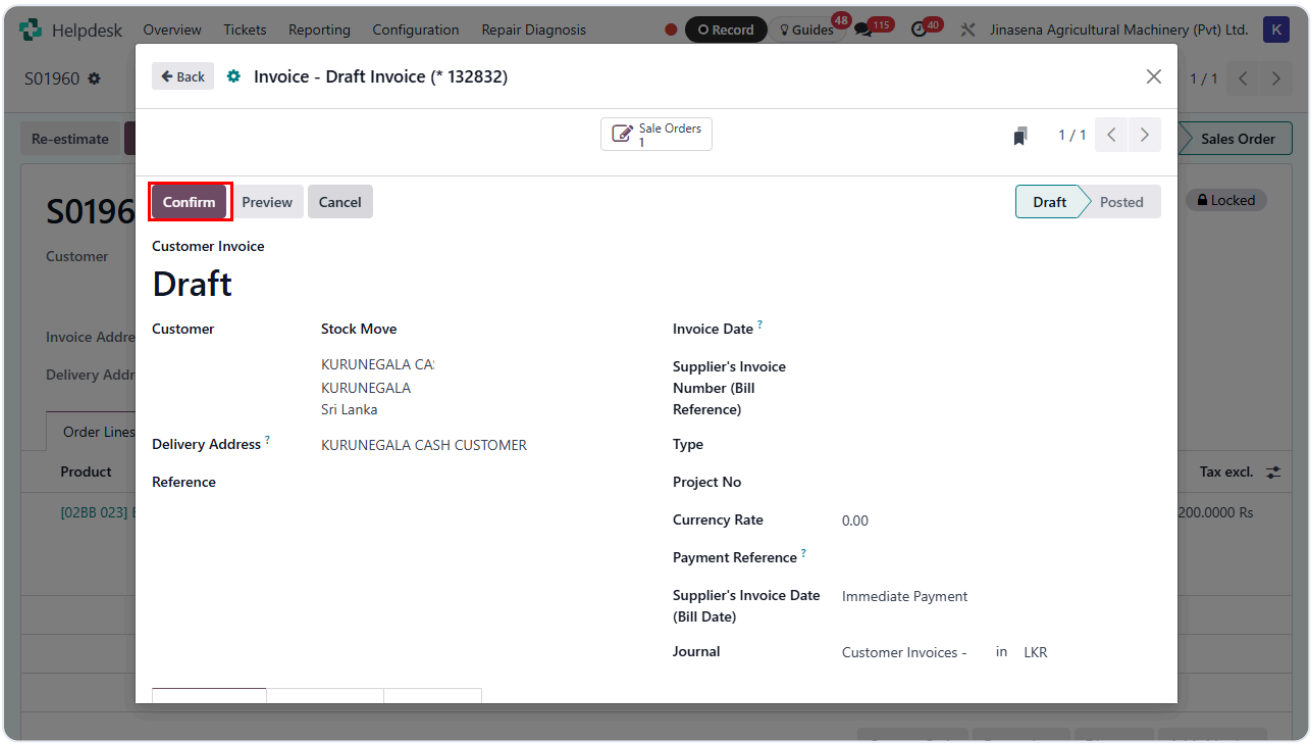
Confirmed Sales Order — Create Invoice button

24 Post the invoice

Navigation Draft invoice

Action **Confirm/post the invoice.**

Description Posting moves the invoice from Draft to Posted so a payment can be registered against it.



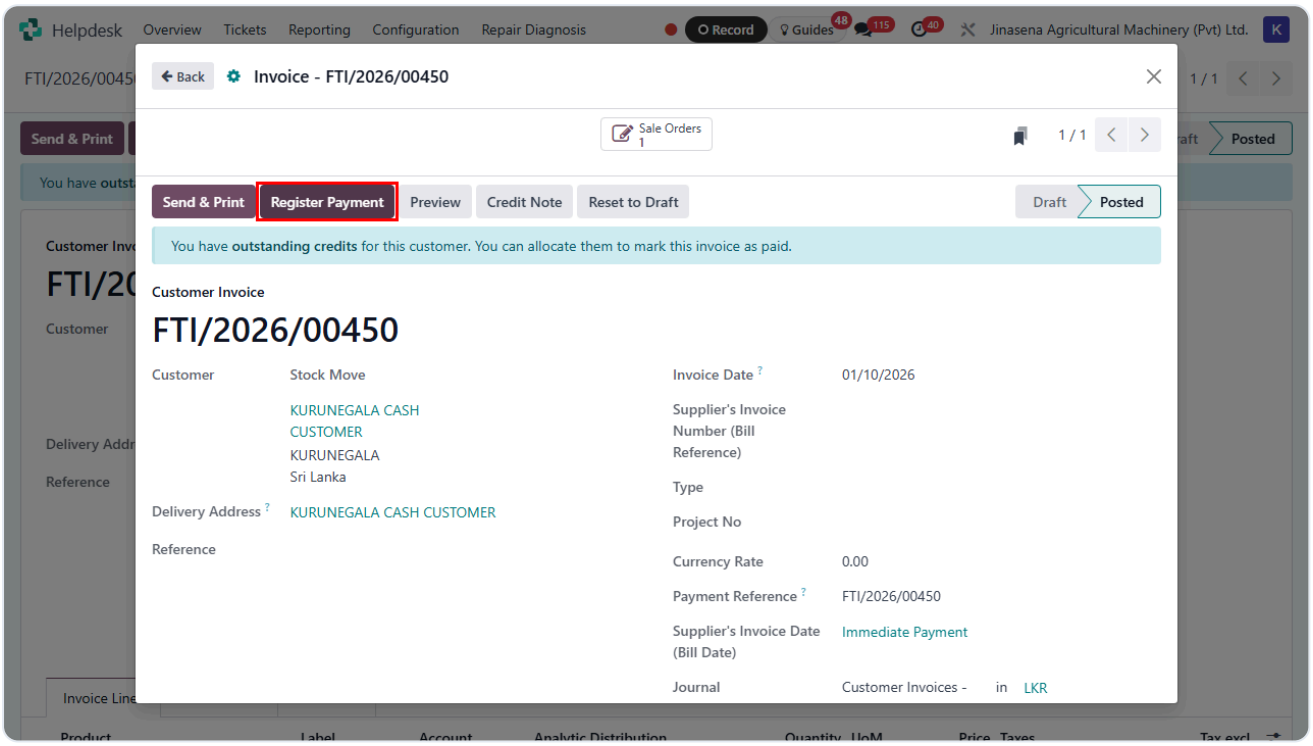
Draft invoice — Confirm button

25 Register the deposit payment

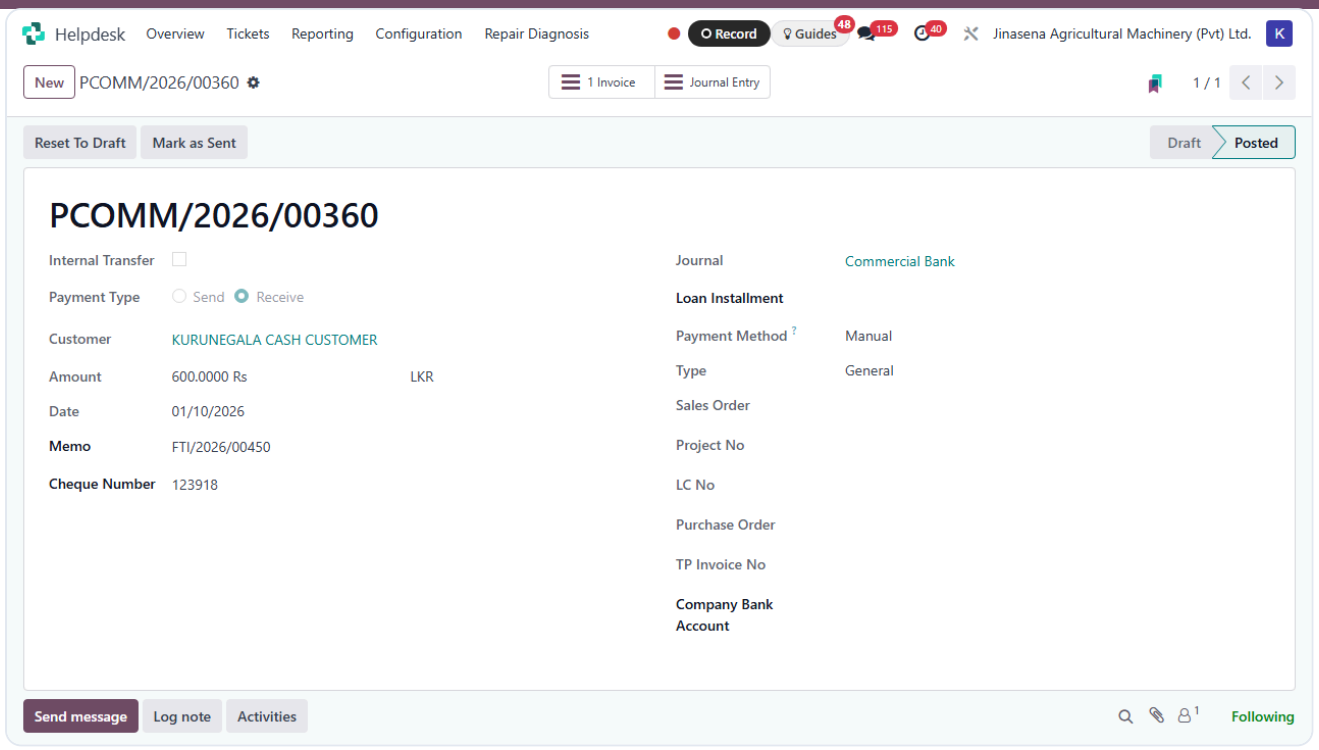
Navigation Posted invoice

Action **Click Register Payment, select Journal: Commercial Bank, enter 50% of the invoice total as the Amount, and click Create Payment.**

Description Registering the payment marks the invoice "PARTIAL" and unblocks delivery validation.



Posted invoice — Register Payment button



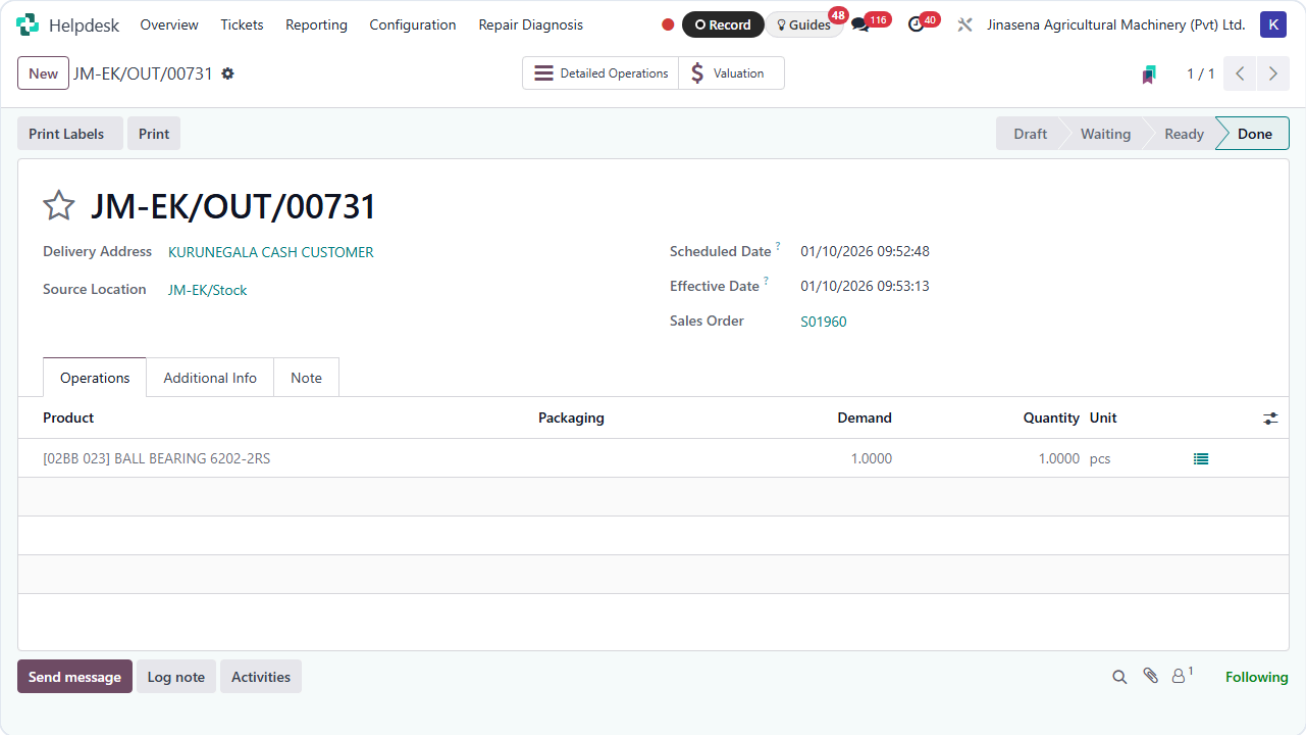
Payment posted, 600 Rs via Commercial Bank

26 Validate the delivery chain

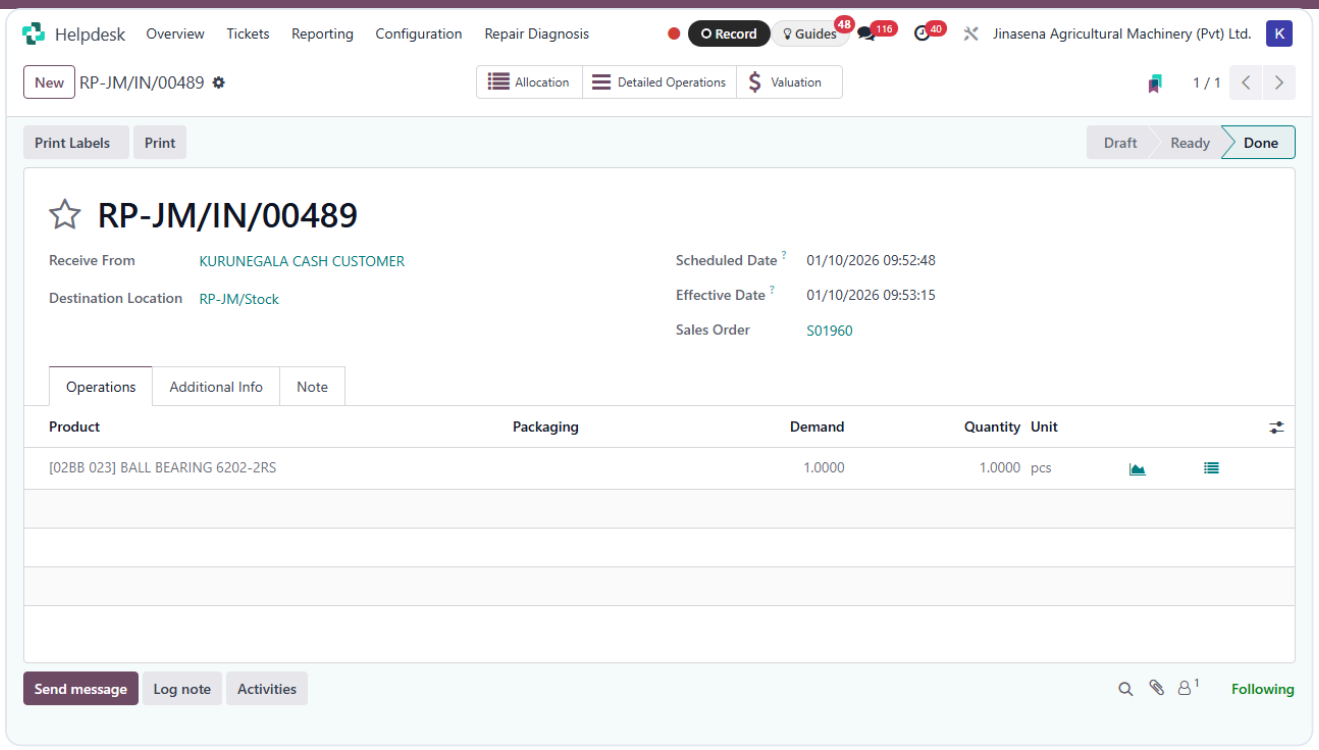
Navigation Sales Order → Delivery stat button

Action **Open and validate the three linked stock moves, in order.**

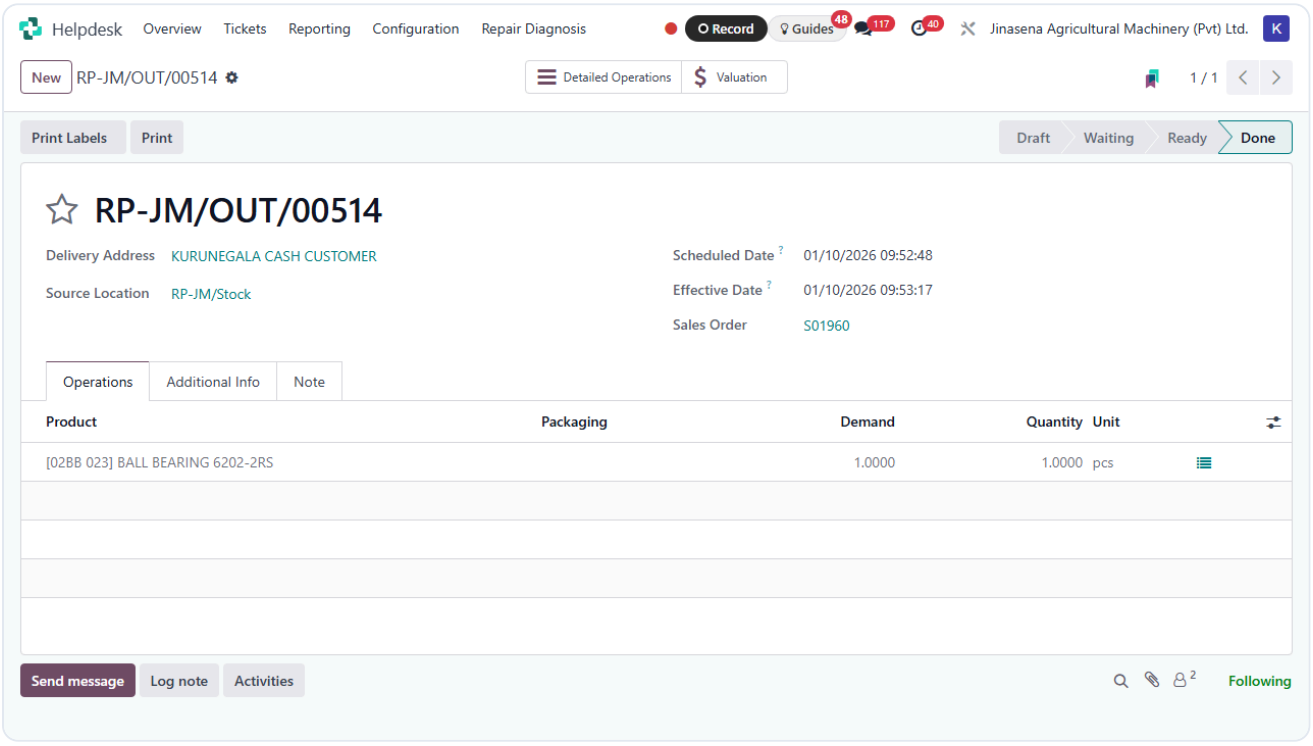
Description Each leg only becomes ready once the one before it is done: outgoing delivery from the resupply warehouse (`JM-EK/OUT/...`), incoming receipt into the repair warehouse (`RP-JM/IN/...`), outgoing delivery to the customer (`RP-JM/OUT/...`). These stock moves are validated via the API directly on this instance — no Validate button is ever shown for them, since they're hidden by the Fix-Repair Studio views like the other accounting/sales actions above.



First delivery leg — Done



Second delivery leg — Done



Final delivery leg to the customer — Done

27 Business rule: deposit before delivery

Navigation —

Action **(informational — no action; do not attempt to validate deliveries before the deposit exists)**

Description Attempting to validate the first delivery before the deposit payment exists fails with a real business-rule error from the Fix-Repair module: "Cannot validate delivery: the customer-pays repair invoice on the linked sale order has no payment yet." This is enforced by Odoo, not just a suggested order of operations.

📷 No separate screenshot for this step — see the text above and the surrounding steps' screenshots for context.

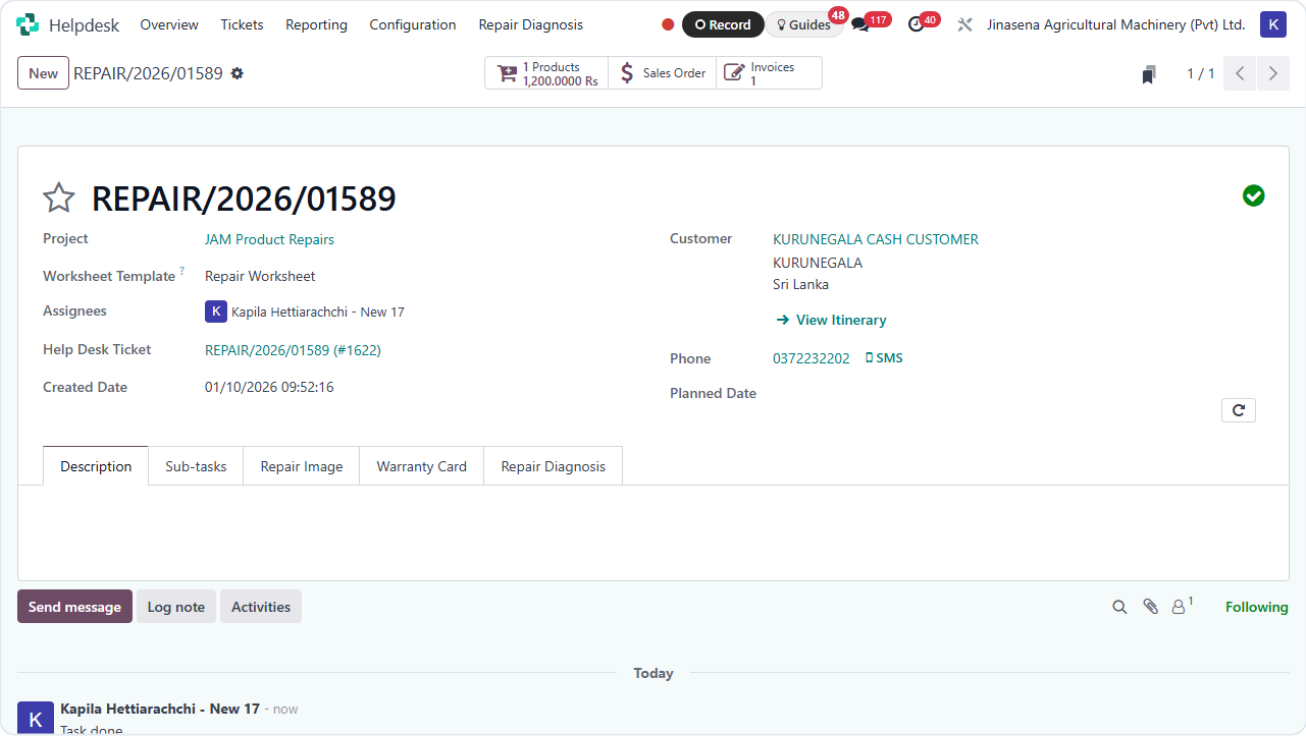
Section F — Complete and Dispatch (Steps 28–33)

28 Mark the task done

Navigation Task form

Action **Click Mark as done in the status bar.**

Description Marks the technician's repair work as complete, moving the task from In Progress to Done.



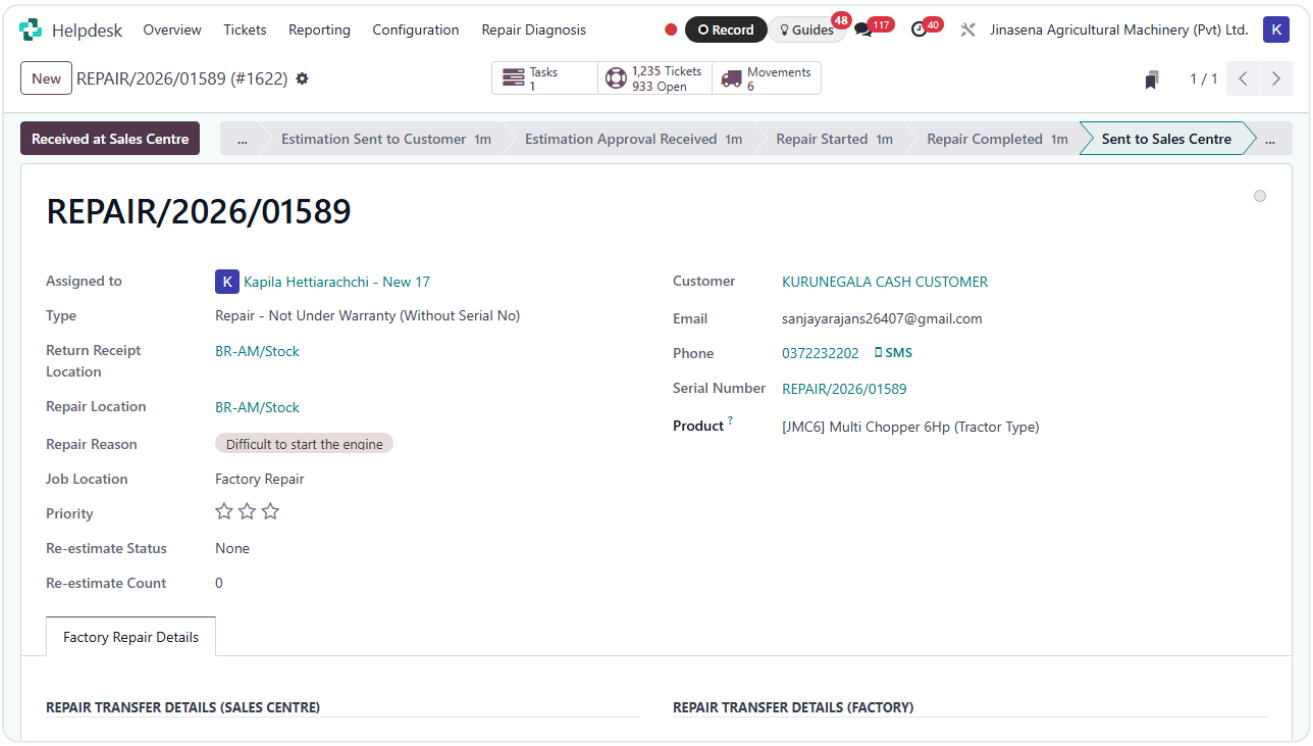
Task marked Done

29 Send to Sales Centre

Navigation Ticket form status bar

Action **Click Send to Sales Centre.**

Description Since the item originated from a sale centre, this routes it back there for hand-over to the customer.



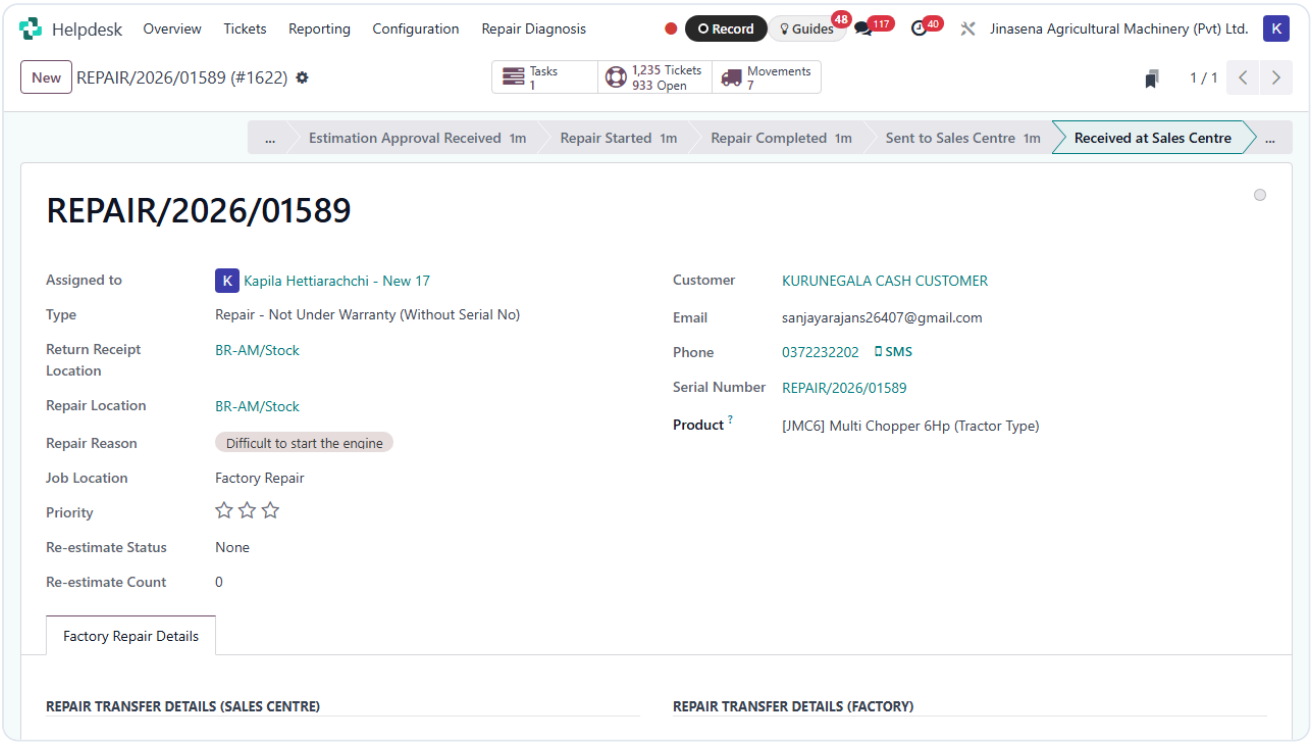
Ticket advanced to Sent to Sales Centre

30 Confirm receipt at the sales centre

Navigation Ticket form status bar

Action **Click Received at Sales Centre.**

Description Confirms the repaired item has arrived at the sale centre.



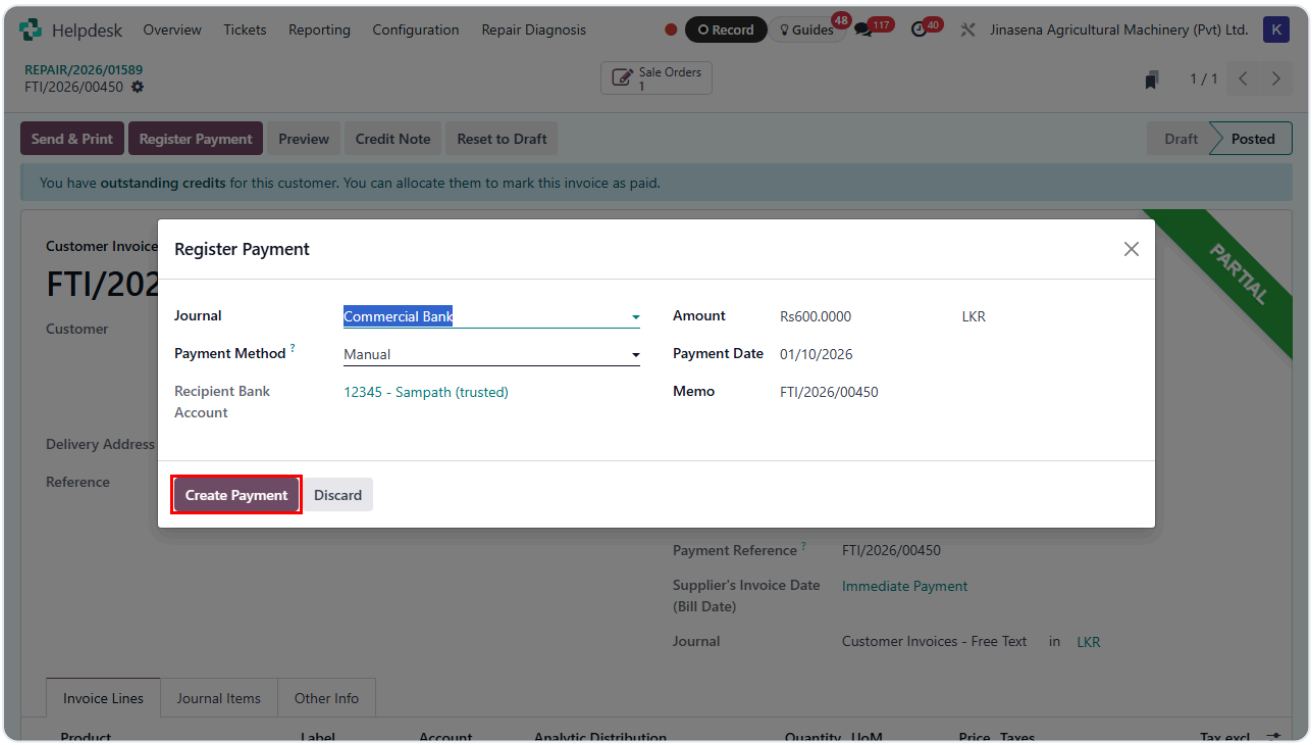
Ticket advanced to Received at Sales Centre

31 Collect the remaining balance

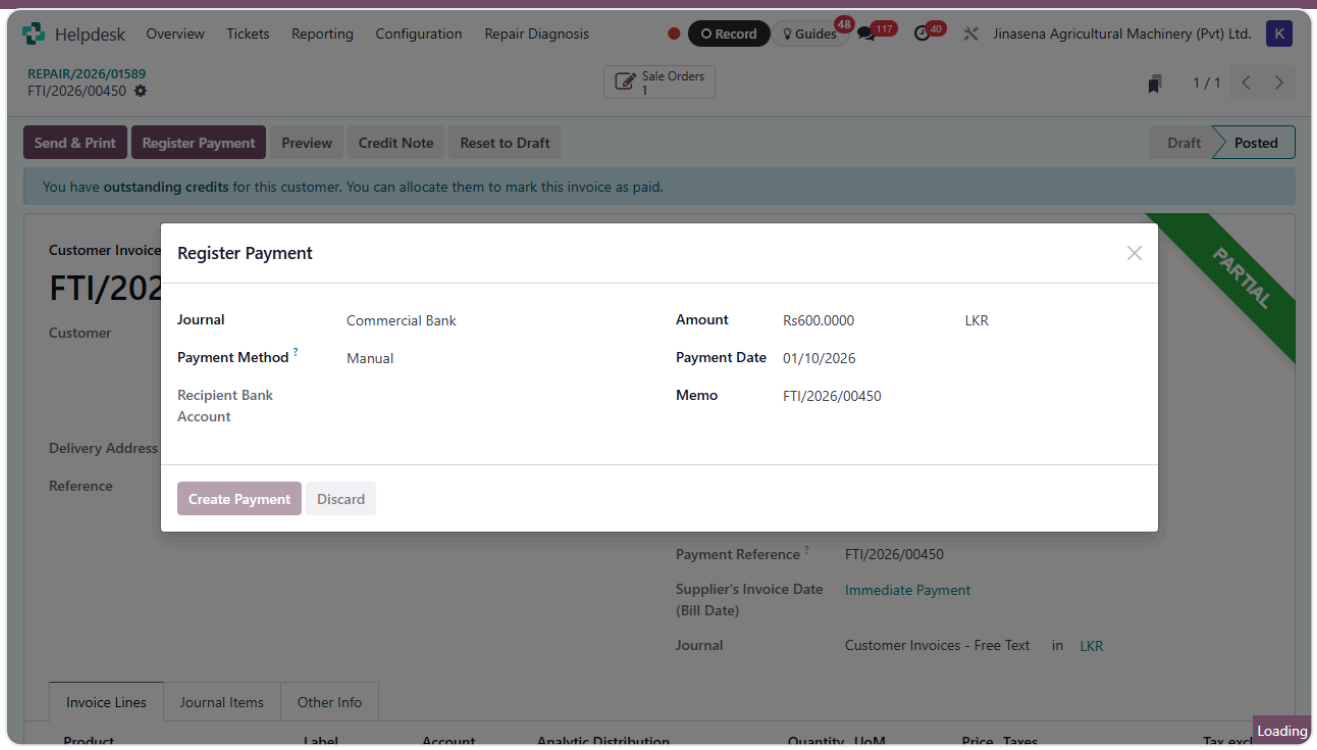
Navigation Task form → Invoices stat button

Action **Open the deposit invoice, click Register Payment, select Journal: Commercial Bank, and click Create Payment.**

Description The invoice now shows a "PARTIAL" banner with its own, non-hidden Register Payment button. The wizard pre-fills the exact remaining balance as the Amount; after payment the invoice flips to "IN PAYMENT". Confirmed live: "Dispatch" visibility can lag the payment by a beat — the automated flow polls with a short reload loop before giving up.



Balance payment wizard filled — Create Payment button



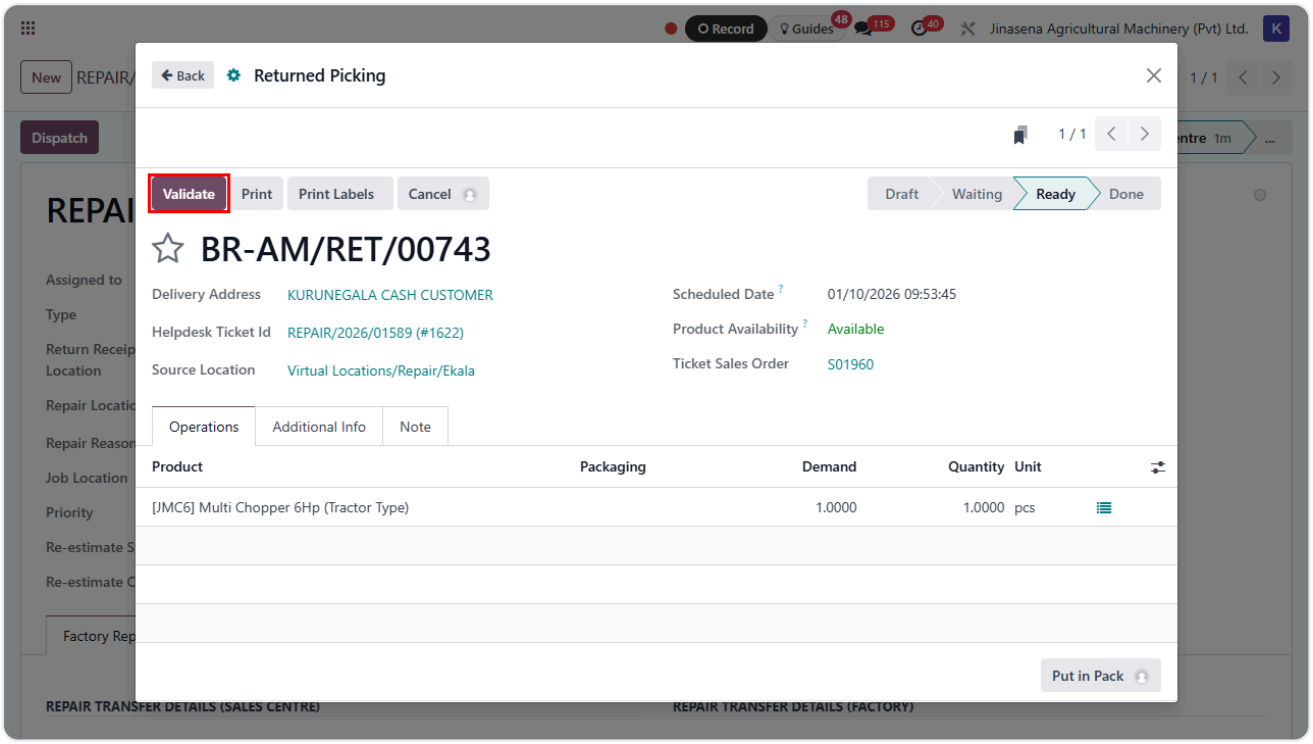
Payment processing — invoice will flip to IN PAYMENT

32 Dispatch the item

Navigation Ticket form status bar

Action **Click Dispatch.**

Description This button only unlocks once the balance is fully paid.



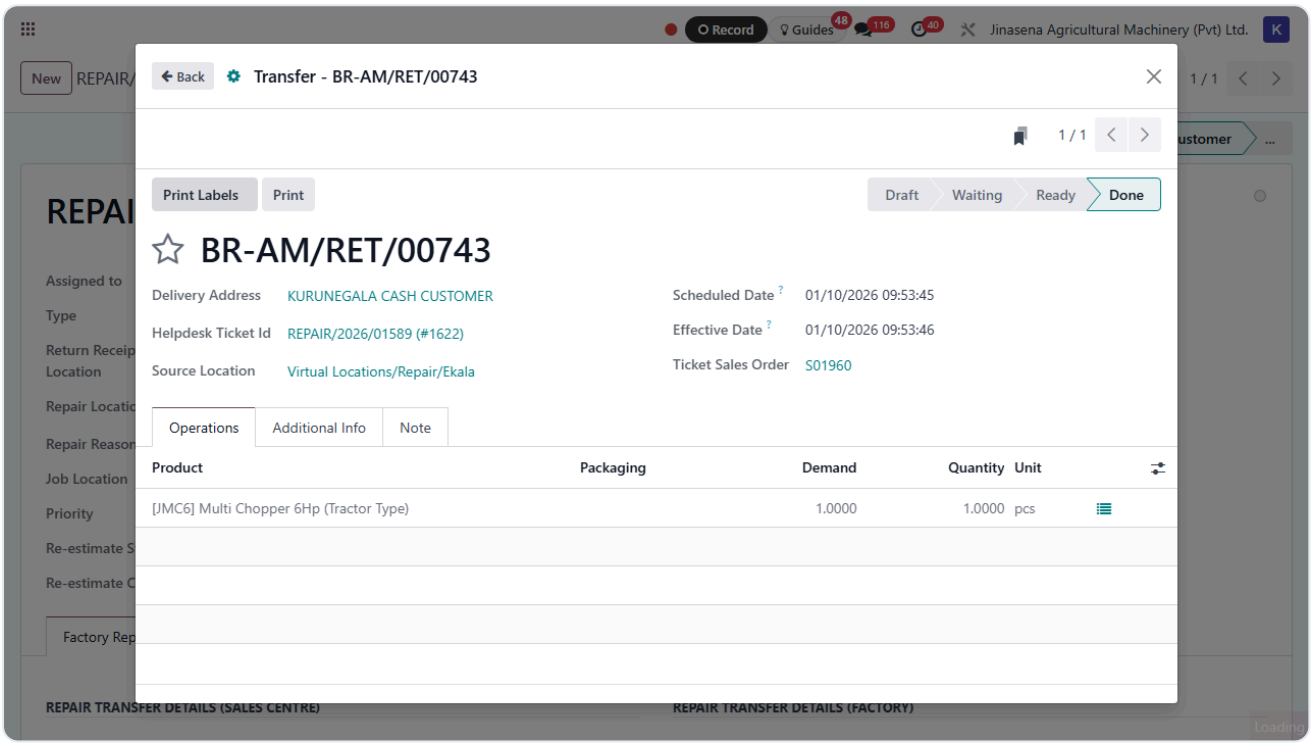
Returned Picking wizard for Dispatch, Ready — Validate button

33 Validate the dispatch

Navigation Returned Picking wizard (Dispatch)

Action **Click Validate.**

Description Closes out the return transfer. The ticket's own status bar now reads "Handed Over to Customer", completing the repair lifecycle.



Dispatch transfer validated, Done state

Notes for QA / SaaS Considerations

- **Implemented: the real online quotation-confirmation flow, not just the RPC shortcut.** Step 22 ("Confirm the Sales Order") now drives the literal path a customer follows — Send by Email → open the emailed "View Quotation" link → public portal page → "Accept & Sign" → "Validate Order" signature popup (pre-filled name and signature) → "Accept & Sign" again — in a real second browser page/tab against the live portal. The flow verifies the order's actual backend `state` via RPC afterward rather than trusting the click sequence, and only falls back to calling `action_confirm` directly if the portal path didn't actually confirm it (e.g. portal access disabled). An earlier version of this closed the portal tab immediately after the final click, which could cancel the signature submission in flight before the backend recorded it — fixed by waiting for the "Validate Order" modal to actually close first.
- **Resolved: the "Return" button was permanently broken, not flaky.** Live RPC investigation traced it to `fix_repair_action_direct_return` always raising `"You may only return Done`

`pickings."` — a genuine bug in the Fix-Repair customization's return wizard. The underlying "Returned Picking" record is still created before that error fires, so the fix calls the method directly (swallowing the expected error) and then validates the resulting picking via RPC — no dependence on the broken wizard UI at all.

- **Resolved: an intermittent procurement/routing race condition on Sales Order confirmation.** `action_confirm` occasionally failed with *"No rule has been found to replenish … in 'Physical Locations/Inter-warehouse transit'"* even though the route and stock were both verified correct — and succeeded immediately on retry with no other change. The automated flow now retries this specific call up to three times before giving up.
- **Resolved: "Dispatch" visibility lagging behind the balance payment by a beat.** The button's computed "fully paid" condition was confirmed correct server-side immediately after payment, but the client needed a reload (or two) to pick it up — the automated flow now polls with a short reload loop instead of a single fixed wait.
- **Resolved: full lifecycle through dispatch, now with real automated screenshots.** Steps 23–33 (deposit invoice → payment → delivery chain → task done → sent/received at sales centre → balance payment → dispatch → final validate) are driven end-to-end by the automated spec, and all of this section's screenshots are captured live from a clean, fully-passing Playwright run — not a manual verification pass. Getting a clean run here required finding and fixing a genuine bug: the invoice opened by "Create Invoice" renders as a dialog on this instance (confirmed even via a fresh, minimal direct-record navigation — not just a stale URL), and left open it silently intercepted clicks on every page rendered afterward, including "Mark as done" much later in the flow. The fix closes it via its own Close button (Escape alone doesn't dismiss it) right after its own screenshot is taken, instead of carrying it forward through the rest of the test.
- **Confirmed live: a second real business rule enforces balance-before-dispatch.** The "Dispatch" status-bar button on the ticket only renders once the deposit invoice's balance is fully paid — clicking "Register Payment" on the task's own Invoices stat button collects the remaining amount (pre-filled by the wizard), after which the invoice flips from "PARTIAL" to "IN PAYMENT" and Dispatch appears.
- **Resolved: quotation → deposit → delivery chain.** The "Confirm" (Sales Order), "Confirm"/post (Invoice), "Register Payment", and "Validate" (delivery) buttons are all hidden on this instance by the Fix-Repair Studio views, even though their underlying model methods (`action_confirm` , `action_post` , `account.payment.register` + `action_create_payments` , `button_validate`) work correctly. The automated flow calls these the same way the Odoo web client itself does internally — via `/web/dataset/call_kw` in the browser's own authenticated session — rather than clicking buttons that don't exist in the DOM.
- **Confirmed live: a real business rule enforces deposit-before-delivery.** Calling `button_validate` on the first delivery leg before any payment exists raises a `UserError` : *"Cannot validate delivery: the customer-pays repair invoice on the linked sale order has no payment yet. Register at least a*

partial payment on the invoice before validating the delivery." This comes from `Fix-Repair/models/stock_picking.py` 's override — it isn't a suggested best practice, it's enforced.

- **Confirmed live: the 3-leg delivery chain is real, not a route-config guess.** Confirming the Sales Order (warehouse RP-JM, resupply route from JM-EK) generates three linked `stock.picking` records in one call: `JM-EK/OUT/...` (JM-EK/Stock → transit), `RP-JM/IN/...` (transit → RP-JM/Stock), `RP-JM/OUT/...` (RP-JM/Stock → Customer). They must be validated in that order — the middle and last legs sit in "Waiting Another Operation"/blocked-on-stock until the one before completes.
- The Sales Order dialog opened via the task's "Sales Order" stat button stays nested inside that dialog's own modal even after changing the URL hash to the order's real id — a stale `.modal.o_technical_modal` layer is left in the DOM and silently intercepts clicks on the "real" page underneath. The automated flow presses `Escape` to close the dialog before navigating to the confirmed order's own form view.
- "Create Invoice" renders as a direct header button when reached from the Sales Order's own full-page view, but as an Action-menu item when reached from the task-opened dialog context — the automated flow tries the direct button first and falls back to the Action menu.
- Reading the invoice's id from the URL right after clicking "Create Invoice" is a race condition — the navigation is asynchronous, and a fixed wait isn't always enough. The automated flow waits for the URL hash to actually contain `model=account.move` before parsing the id out of it.
- Steps 1–22 use real reference data confirmed to exist on this instance (see `Data.txt` / `FIX_REPAIR_TEST_CONFIG` in `fix_repair.master-data.ts`): customer `KURUNEGALA CASH CUSTOMER` , product `[JMC6] Multi Chopper 6Hp (Tractor Type)` (searched by its short code `JMC6` — searching the full bracketed display name was confirmed server-side but proved unreliable client-side), repair reason `Difficult to start the engine` , the exact Repair Diagnosis values, and spare part `[02BB 023] BALL BEARING 6202-2RS` (has stock on this instance), located via the catalog's own search box.
- The Repair Image field renders as two separate slots (`x_studio_repair_image_01` / `_02`) each with their own hidden file input — a generic `input[type="file"]` first-match grabs an unrelated hidden input elsewhere on the page (e.g. chatter attachments) rather than either image slot; the automated flow scopes the selector to the first image field's own container.
- **Resolved: "Products" step.** Earlier attempts concluded this button (`action_fsm_view_material`) was permanently absent from this instance's backend Task form. That was wrong — it renders as a stat button reading "0 Products / Rs0.0000" in the header button box, but **only once both Repair Diagnosis and Repair Image are set** (the same "Add Data" banners that nag for those two fields). Before that point it genuinely isn't in the DOM at all, which is what caused the earlier false negative.
- Confirmed live: `ticket_type_id` and `x_studio_return_receipt_location` are many2one autocomplete fields, `x_studio_job_location` is a plain HTML `<select>` , and `x_studio_repair_reason` is a many2many tags field. The ticket's own title (`name`) is not user-

editable — it's assigned via sequence the moment the ticket is first saved, which happens as a side effect of clicking **Assign to Me**.

- This instance's many2one/many2many autocomplete dropdowns render with newer `o-autocomplete--dropdown-menu` / `o-autocomplete--dropdown-item` CSS classes rather than the older `o_field_many2one_dropdown` ones — the shared `Many2OneField` / `Many2ManyField` page-object components were updated to recognize both.
- Selecting Customer triggers an onchange re-render that can briefly leave Product's input inert if interacted with immediately — the automated flow waits ~1s after each such field commit.
- **Create Serial No** only takes visible effect once the ticket has no pending unsaved changes — the automated flow does an explicit save first if needed.
- The top navbar has a "Tickets" menu item with the exact same accessible name as the dashboard card's own "Tickets" button — a generic role-based lookup resolves to the navbar one (which opens a "My Tickets" / "All Tickets" dropdown) rather than the card. The automated flow goes through that dropdown (→ "All Tickets") directly instead of fighting the ambiguity.
- The tour export's recorded trigger for "Return" was a generic `[data-hotkey="q"]` selector, which isn't present as an attribute on this instance's status-bar button — the automated flow matches it by its visible label instead. "Validate" (`data-hotkey="v"`) inside the stock transfer wizard did work as recorded.
- The "Create & View Task" flow opens the Field Service task in a dialog on top of the ticket (not a full-page navigation) — waiting on a generic "dialog visible" condition resolves immediately against the still-open "Create a Field Service task" dialog itself; the automated flow instead waits for that dialog's title to disappear.
- Closing the product catalog navigated all the way back to the ticket (not just the task) — the generic "close" selector used matched a breadcrumb/dialog-stack close rather than only the catalog's own close button. The Sales Order smart button appears once the task is reopened via its own "Tasks" stat button (resolved — see next note).
- The ticket form's stat buttons (Tasks/Tickets/Movements) use the class `o-form-buttonbox` (hyphenated) rather than the task form's `o_button_box` (underscored) used by the Products button — and the DOM briefly holds two copies of the same button during a re-render, so the automated flow scopes to `button.oe_stat_button:visible` rather than a plain `.o_button_box` selector when acting on ticket-level stat buttons.

Generated from the Odoo Tour Recorder export · fix_repair module · Markdown version